RESPONSE TO REVIEW

RaspFormer

Point-by-point answers to the external review

This document answers every point in the review, in the reviewer's order. For each point it gives our verdict, what the evidence shows, what changed after the review, and what is still open. Every number from the repository was recomputed from raw files on 17 October 2026; literature figures were checked against the papers. The findings were re-checked by a separate verification pass that tried to refute them, and this text by a final fact-check. Corrections from both are included.

Short answer to the headline critique

The reviewer is right. The original study is a **compiler-fidelity check**. It shows that Tracr's compiled residual stream matches independently evaluated RASP values. It does not test whether PR or CKA can reveal computation.

The research upgrade added basis controls, blind write detection, geometry-guided patching, compression and trained models. Re-analysed here, these experiments explain the problem more precisely but do not resolve it.

In compiled models, each stage's geometry is fixed by which values have been written. The blind detectors only find sublayers that are non-empty, which can be read straight from the weights. Whether PR or CKA can reveal computation is still an open question. Section 16 lists the experiments that could answer it.

25	22	2	1
review points answered	agreed with in full	agreed with in part, or refined	disagreed with: the P12 failing-set inference

Three versions of the work are referred to throughout

Label	Meaning	Where
Original	The committed study the reviewer read (Steps 1-5).	Commit `2dc9118`. Report: `git show 2dc9118:results/report/RaspFormer_Steps_1_to_5.md`
Upgrade	Research upgrade, experiments recorded on 15 October 2026 and committed on 17 October 2026: variance floor, controls, blind detection, patches, compression, 36 trained models.	Commit `e06479c` on branch `raghulchandramouli/repo-overview` of github.com/raghulchandramouli/RaspFormer (not yet merged into main). Evidence: `results/<study>/reference/`. Its manifests were written before the commit, so they record `2dc9118` with `git_dirty = true`.
New	Analysis done for this response on 17 October 2026. Not part of the repository's results.	`.context/review-response/<topic>/`: local and gitignored, so not in the pushed repository; available on request.

Where to start

Section 1 gives a one-line answer and status for all 25 points. **Section 2** answers the headline critique. **Section 15** lists every claim we withdraw or correct. **Section 16** ranks the experiments that could still answer the headline question.

Contents

1 Summary of answers . . . 3
2 The headline critique . . . 5
3 Issue 1: the benchmark is near-circular . . . 6
4 Issue 2: numerical artifacts . . . 9
5 Issue 3: P12 lane errors . . . 12
6 Issue 4: the complexity analysis . . . 15
7 Issue 5: interventions . . . 17
8 Issue 6: transfer limits . . . 19
9 Next step 1: random orthogonal rotation . . . 20
10 Next step 2: compression and superposition . . . 21
11 Next step 3: trained models . . . 23
12 Next step 4: blind recovery of the write schedule . . . 24
13 Positioning against InterpBench . . . 26
14 The parts you liked: what survived . . . 27
15 What we withdraw or correct . . . 28
16 Experiments that could answer the headline question . . . 28
Appendix A Reproduction checks run for this response . . . 29
Appendix B Evidence index . . . 29
References . . . 30

1 Summary of answers

Verdict says whether we agree with the reviewer. Status says where things stand after the upgrade and this response. **Done** means the requested work exists and was checked. **Partly done** means some of it exists. **Open** means it does not exist yet. **Conceded** means we accept the point and nothing fixes it. **Withdrawn** means we retract the original claim. **Explained** means the question is answered and no change was requested. Statuses that rest on analysis done for this response are marked *new analysis*.

#	Reviewer point	Verdict	Status	Answer in one line	Sec
H	Mostly confirms Tracr compiles correctly; does not test whether PR or CKA reveal computation	Agree	Open	Correct. In compiled models, geometry is fixed by what has been written. The detectors only find non-empty sublayers.	2
1.1	Expected representation = compiled residual minus scratch lanes	Agree	Conceded	Exactly right. Only the values are independent. The 9 SelectorWidth scratch lanes are zeroed.	3
1.2	Gaps can only come from scratch lanes, approximation, or near-zero variance	Agree	Done	Tested exhaustively. Every gap falls into one of the three, with none left unexplained.	3
1.3	PR tracks value count (k-1), not the algorithm	Agree	Done	Exact closed form for single-variable programs. Under balanced probes, three algorithms with 6 values all have expected PR 5.000.	3
1.4	PR error is a poor headline metric	Agree	Partly done	One varying lane against an empty reference gives a gap of exactly 1 at any variance. The upgrade demotes PR error but still lists it first.	3
2.1	P07/P08/P10 at exactly 0.5 = PR on near-zero noise	Agree	Done	Three of six stages gave PR 1 vs 0 on 8e-19 variance: a one-ulp split in the 74-row final batch.	4
2.2	The 14 CKA availability mismatches are the same thing	Agree, refined	Done	12 of 14 were. The other 2 (P05, P06) are real scratch-lane variance and remain.	4
2.3	Save total variance, set a floor, rerun everything	Agree	Done	Done. Floors from 1e-14 to 1e-8 give identical results. The root cause is still in the code, and the margin against known numerical error is small.	4
3.1	87.5% in all 8 lanes = the same 1/8 fails everywhere	Disagree	Done	The failing sets are disjoint. Each lane fails exactly where it should read 1.	5
3.2	Cause: softmax not saturating on ties or duplicates	Partly agree	Done	Non-saturation, yes. Input ties, no. Logits are divided by sqrt(key_size = 50), set by the sort's tie-breaking keys (new analysis).	5
3.3	Find failing rows, sharpen attention, check the error shrinks	Agree	Done	Error 8.5e-4, 7.2e-7, 1.4e-24, 0 at scales 1, 2, 4, 8. Sharpening only layer 3 is enough.	5
4.1	Why does variable count equal depth?	Agree	Explained	Every program is a single dependency chain (DAG width 1).	6
4.2	Two of three axes are collinear; no conclusion possible	Agree	Withdrawn	r = 1.00 and 0.98, with 4 distinct points. We withdraw the complexity analysis.	6
4.3	Need programs that vary depth and width	Agree	Partly done	Requested depth and branches are orthogonal, but Tracr depth and count still correlate (r = 0.96), and geometry is a closed-form function of what is written.	6
5.1	A patches the output directly; trivially correct	Agree	Conceded	Withdrawn as causal evidence. 7 of 12 upgrade patch sites are also readout sites.	7
5.2	B is one dependency, with donors picked to flip	Agree	Partly done	Upgrade donors are drawn without looking at outcomes, but only 15 of 192 stored pairs test a downstream step.	7
5.3	Neither connects to the Step 3 geometry	Agree	Partly done	The upgrade picks the stage by 1-CKA and the subspace by PCA, but PR is unused and stage choice is nearly trivial.	7
5.4	Patch PCA directions or high-CKA transitions; map to variables	Agree	Partly done	Stored pairs: PCA succeeds only at readout sites (0/15 downstream). A new P09 test gives weak, one-directional downstream evidence. High-CKA patching not run.	7
6.1	No superposition, no erasure, axis-aligned lanes	Agree	Partly done	Confirmed: lanes never change after their write. Rotation (N1) and untrained compression (N2) cover parts; erasure is open.	8

#	Reviewer point	Verdict	Status	Answer in one line	Sec
6.2	Never overwritten, so lifetime effects cannot appear	Agree	Open	No experiment tests erasure or lifetime. Tests are proposed.	8
N1	Random rotation: PR and CKA hold, lane checks break	Agree	Done	Whole-residual PR and CKA change by at most 1.4e-13 (guaranteed). Lane agreement, recomputed here, falls from 0.996 to 0.0003.	9
N2	Compress to induce superposition, then retest	Agree	Partly done	The projection is untrained, not learned superposition. With a random basis, accuracy falls to 0.376 while CKA-detector F1 rises.	10
N3	Train or fine-tune models; compare to compiled	Agree	Partly done	36 models trained. P11 was not reliably learned. PR is 2.4-17x lower. CKA is no better than at initialization in 6 of 12.	11
N4	Blind recovery of the write schedule	Agree	Open	No. Only 'non-empty sublayer' is recovered, which can be read from the weights.	12
N5	Position against InterpBench	Agree	Partly done	Positioning written and an integration experiment designed; not run.	13

Points the reviewer liked (reproducibility, honest limits, Fig. 9, full vs computed views) are covered in Section 14. Section 15 lists every claim we withdraw or correct.

Terms used throughout

Term	Meaning
Participation ratio, PR	(sum of covariance eigenvalues)^2 / sum of squared eigenvalues. An effective dimension that does not depend on scale.
Linear CKA	Centered kernel alignment between two sets of activations for the same observations. It is invariant to rotation and to uniform scaling.
Expected representation	RASP values evaluated independently of the model, placed in the compiler's lanes at the compiler's write times.
Full / computed view	Full uses every residual lane. Computed drops token, position and constant lanes but keeps compiler scratch (helper) lanes.
Matched / balanced design	Matched: the same 3,383 sequences x 6 positions = 20,298 observations for every program. Balanced: 64 observations per reachable output class.
Stage names	`Embedding`, then `L1/Attn`, `L1/MLP`, `L2/Attn` and so on: the residual stream after each sublayer.

2 The headline critique

H Does the study test whether PR or CKA can reveal computation?

> *"Strong execution, but it looks like it mostly confirms Tracr compiles correctly tbh. It doesn't yet test whether PR or CKA can reveal computation."*

Verdict Agree **Status** Open

Answer. Correct, and the upgrade does not change it. The original report said its scores measure *fidelity to this explicit benchmark* and that the study *does not independently show that a metric can discover computation in an unknown trained network* (original report, line 504). But it still described the project as evaluating representation measurements against known computations in Tracr (line 511) and led with PR error, which made it read as more than a fidelity check. Re-analysing the upgrade's new experiments gives five concrete reasons why the compiled setting cannot answer the question as posed.

- **Geometry is fixed by what has been written.** In Tracr, each variable owns lanes that no other variable uses. They are exactly 0 before the variable is written and never change afterwards (52 of 52 variables, Section 8). So a stage's PR and CKA depend only on which value distributions have been written so far. In the upgrade's depth-by-branch family this holds exactly: a closed form (new analysis) reproduces all 63 stage PRs to within 1e-13 (Section 6).
- **PR mostly measures the value distributions of the written variables.** A formula that uses only the value frequencies of every written variable explains R^2 = 0.95 (matched; 0.88 balanced) of final-stage PR across the 12 programs. Under class-balanced probes, three different algorithms with six output values all have expected PR = 5.000; on matched inputs they differ only through value frequencies (4.55, 2.99, 4.51) (Section 3).
- **Blind detectors find non-empty sublayers.** Sublayers that write nothing change the residual by exactly 0. A rule that uses only the weights ('is this sublayer's output projection non-zero?') matches the best detector at 54 of 54 transitions (Section 12).
- **Detector scores can improve while the computation breaks.** Projecting the residual onto a random half-width basis cuts held-out token accuracy to 0.376. Yet the CKA detector's F1 rises from 0.850 to 0.879 (Section 10).
- **Where nothing is compiled, there is no ground truth.** Trained models have no known write schedule. On them, the frozen detectors, run on them for this response, flag 138-144 of 144 transitions (Section 12).

What we can and cannot claim

Can claim: a careful, reproducible compiler-fidelity benchmark with an explicit numerical policy. It comes with controls that show *where and why* PR- and CKA-based write recovery succeeds only trivially in compiled models (it detects non-empty sublayers) and does not track the computation.

Cannot claim: that PR or CKA reveal computation. Answering that needs models with known internal structure whose lanes are not axis-aligned, not append-only, and whose sublayers are not trivially empty. Examples are InterpBench's SIIT models, compiled models fine-tuned end to end, and compiled models that erase and reuse lanes. Learned compression adds superposition but leaves empty sublayers empty (Section 16).

3 Issue 1: the benchmark is near-circular

1.1 Expected representation = compiled residual minus scratch lanes

> *"Benchmark is near-circular. Expected representation = compiled residual minus scratch lanes."*

Verdict Agree **Status** Conceded

Answer. Exactly right. Only the variable *values* are computed without the compiler, by the RASP evaluator. The encoding, lanes, write times and persistence all come from Tracr. The only lanes the expected representation leaves out are 9 SelectorWidth attention-output scratch lanes: one each in P05-P10 and P12, two in P11, and none in P01-P04. Apart from numerical error and P12's attention leak (a property of the assembled weights, Section 5), the expected representation *is* the compiled residual with those lanes set to zero. The original report disclosed this construction ('Unwritten and scratch lanes are zero', lines 285 and 486) but did not draw the circularity conclusion. The upgrade does not change how it is built.

Ingredient	Source	Independent of the compiled model?	Code
Variable values	RASP `DefaultRASPEvaluator` on probe tokens, read at the target position	**Yes**	`alignment.py:42-51`
Encoding	Compiler `OUTPUT_BASIS`: one-hot for categorical, scalar for numerical, 0 for None	No	`alignment.py:23-33, 51`
Residual lanes	Compiler `OUTPUT_BASIS` labels	No	`compiler.py:115-121`
Write time	Tracr layer allocation; inserted at first write + 1; SelectorWidth moved to its MLP	No	`compiler.py:103-108, 144`; `alignment.py:69-72`
Persistence	Never cleared after writing (as in Tracr)	No	`alignment.py:66-75`
Scratch lanes	Never filled, so they stay 0	-	(absent)
Views	Full = all lanes; computed = all minus tokens/indices/one (scratch kept)	-	`geometry.py:292-301`

What it can and cannot detect. Because the values are independent, the benchmark would catch a compiler that writes wrong values, writes at the wrong time, or mis-embeds inputs. It found none of these. Input lanes and named lanes before their write match exactly (difference 0). Named lanes after their write differ by at most 1.9e-6, except P12's sorting output (8.5e-4, Section 5). So this is a legitimate fidelity test, and the upgrade labels it that way (*compiler fidelity; distinct from blind recovery*, comparison manifest).

As a test of whether PR or CKA reveal computation, though, it is circular. 'PR error' simply measures PR(compiled) minus PR(the same residual with scratch lanes zeroed and numerical error removed).

Still open: no experiment uses ground truth that does not come from Tracr's own layer allocation.

1.2 Gaps can only come from scratch lanes, approximation, or near-zero variance

> *"So gaps can only come from scratch lanes, approximation error, or near-zero variance."*

Verdict Agree **Status** Done

Answer. We tested this exhaustively and it holds, with nothing left over. We split the difference (observed minus expected) into four lane groups: inputs, written named lanes, unwritten named lanes and scratch lanes. We then measured how much each group alone changes PR and CKA. Summed together, the single-group effects reproduce every gap to within 0.23%. Any gap outside the three categories would have to be a wrong named value, a timing error or an input error. None occurred. The categories also overlap: the near-zero-variance artifact sits *inside* a scratch lane (the constant `length` helper).

Gap type (upgrade, matched design)	Computed view	Full view	Cause
PR gap > 1e-6	19 (16 scratch, 3 approx.)	20 (16 scratch, 4 approx.)	Scratch lanes in P05, P06, P09, P11, P12; float32/MLP approximation in P07, P08, P11
CKA error > 1e-9	16 (8 scratch, 7 approx., 1 mixed)	20 (12 scratch, 7 approx., 1 mixed)	Mixed = P12 L3/Attn: scratch 0.027 + named 0.0005
CKA availability mismatch	2 (scratch)	0	P05, P06: helper lane varies before the count is written
Original only, no floor	12 PR gaps of exactly 1.0; 12 mismatches	-	Near-zero variance (P07, P08, P10, P11 x 3 stages)
Lane value outside tolerance	16 lane rows (the lane check does not depend on the view)	same 16 rows	P12's 8 output lanes at L3/Attn and L3/MLP: attention leak (Section 5)
Unexplained	**0**	**0**	Input-lane difference 0; pre-write named-lane difference 0

The balanced design has the same structure. Helper program B adds 4 PR gaps per view, 1 CKA error (computed) or 3 (full), and 1 computed-view mismatch, all from scratch lanes; A has none. Size: P12's matched computed PR error of 2.134 is 2.132 from scratch lanes and 0.002 from approximation. Its largest stage gap (-7.92 at L2/Attn) has no named-lane part at all. Source: `circularity-pr/gap_attribution_*.csv`, re-checked in `circularity-pr/verify/`.

1.3 PR tracks how many values a variable takes

> *"PR tracks how many values a variable takes (k-1), not the algorithm. P01, P02 and P03 give 5, 1 and 12."*

Verdict Agree **Status** Done

Answer. Confirmed, and exact for single-variable programs. Take a one-hot code with class frequencies p, centered. Its PR is $(1 - \text{sum } p^2)^2 / (\text{sum } p^2 - 2 \text{ sum } p^3 + (\text{sum } p^2)^2)$. With balanced classes this is exactly k - 1. Using class counts alone, the formula reproduces the final PR of P01-P03 to 6 decimals: 5, 1 and 12 (balanced) and 4.548, 1 and 9.697 (matched). P01 (an elementwise map), P05 (a histogram via SelectorWidth) and P06 (a comparison count) are three different algorithms with k = 6. Under balanced probes all three have expected PR 5.000, because the sampler forces equal class frequencies; on matched inputs they differ only through their value frequencies (4.548, 2.992, 4.510).

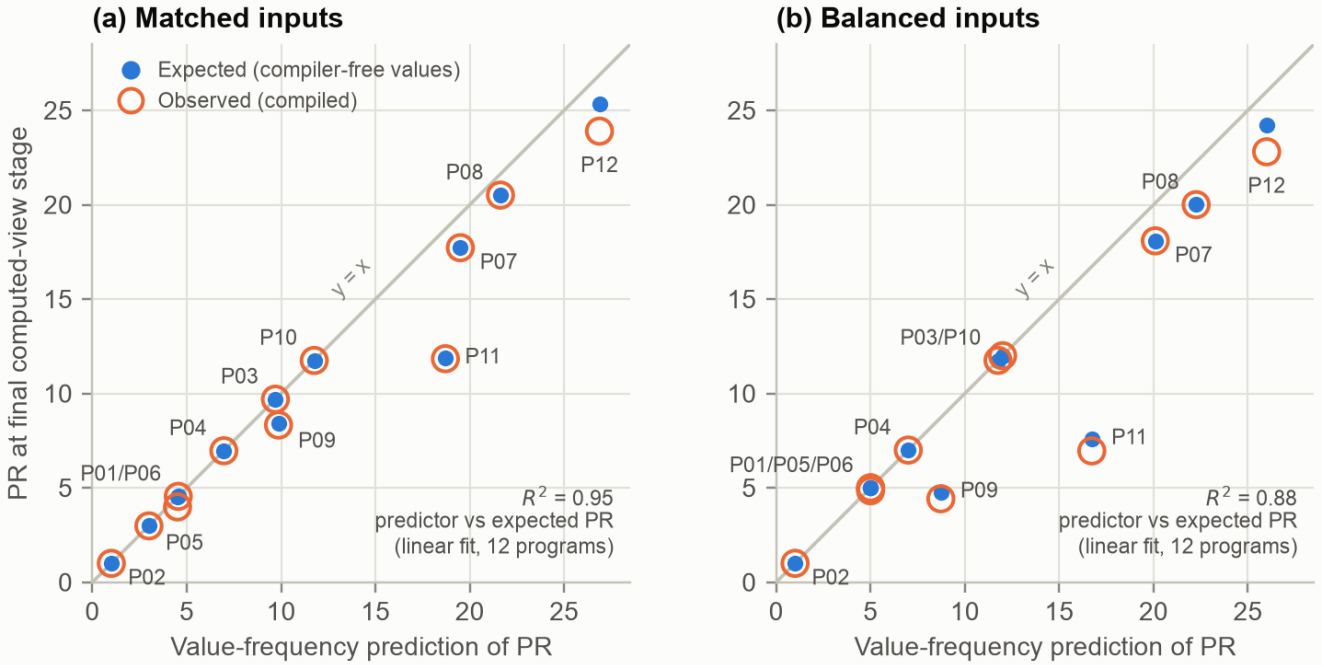


Figure 1. Final-stage computed-view PR against a prediction that uses only the value frequencies of every written variable, treating each variable's one-hot block as uncorrelated. Filled markers: expected representation. Hollow markers: compiled model. Expected values for single-variable programs sit on the identity line; the compiled P05 and P06 sit below it because of their scratch lane. Every multi-variable program except P10 falls below the line, most of all P11 and P09, because their intermediates are functions of one another. Source: `circularity-pr/fig1_pr_vs_cardinality.csv` (upgrade data, new analysis).

Across programs, the value-frequency predictor explains R^2 = 0.95 (matched) and 0.88 (balanced) of final-stage expected PR. Without the high-leverage P12, it explains 0.93 and 0.83.

Two qualifications keep this from being overstated:

- **Output cardinality alone is a weak predictor.** k_{out} - 1 gives R^2 0.32 (matched) and 0.52 (balanced). The high R^2 needs the value distribution of every persistent intermediate variable, which depends on how the program is broken into steps.
- **Final PR sums categories over every variable written.** Tracr never erases, so final PR roughly adds up the effective categories of every variable in the program. That reflects how the program is broken into steps. It says nothing about how each value is computed.

Under balanced probes, PR = k - 1 for single-variable programs is also forced by the sampler, which draws 64 per class. The original report already stated the k - 1 identity with this same 5 / 1 / 12 example (original report, lines 271 and 357), but still put PR error in the first column of its headline table.

1.4 PR error is a poor headline metric

> *"PR error is a poor headline metric for that reason!"*

Verdict Agree **Status** Partly done

Answer. Agreed. The case is stronger than the reviewer states. We drop PR error as a headline in this response. The upgrade's generated report already calls PR agreement secondary and moves the fidelity table to a late section (`research-validation.md:205-207`), but that table still lists PR error in its first metric column (`report.py:195`), and that should change. PR error has four separate defects.

- **Its baseline is cardinality** (point 1.3).
- **It has no scale.** Any single varying direction compared with an empty reference gives a PR gap of exactly 1.0. That holds whether the variance is 8e-19 (the P07 noise in the original) or 0.11 (the P06 helper lane in both studies); see Figure 2. This one effect is 99% of P05's PR error and 66% of P06's, and the variance floor cannot remove it.
- **Its sign flips for a single cause.** One P09 scratch lane gives +0.272 at L2/Attn and -0.077 at L3/MLP. P11 at L5/Attn gives +0.132 with matched inputs and -0.313 with balanced inputs.
- **Averaging distorts it.** One stage can dominate: P12's L2/Attn is 62% of its error. The number of transitions also dilutes it: the same three artifact stages give 0.5 for P07 (6 transitions) but 0.25 for P11 (12). The size of the PR gap is not proportional to how much of the representation is unexplained (Pearson 0.16 matched, 0.29 balanced); even in rank it agrees only partly (Spearman 0.85 matched, 0.39 balanced, on stages with a real gap).

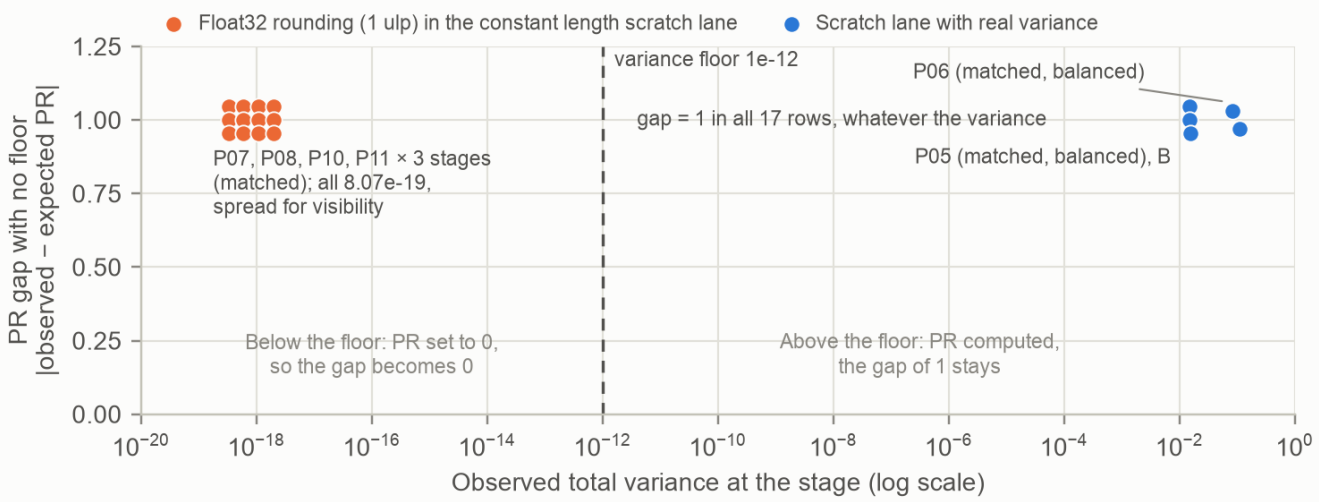


Figure 2. PR gap with no variance floor, plotted against observed total variance. The 17 computed-view stages shown are those where the expected representation is empty but the observed residual varies: the 12 matched artifact stages, P05 and P06 at L1/Attn in both designs, and helper program B at L1/Attn. Every gap is exactly 1, whether it comes from one-ulp noise (8e-19) or real helper-lane variance (0.015-0.11). The 1e-12 floor only decides which gaps survive: below it, PR is set to 0 and the gap becomes 0; above it, the gap of 1 stays. Points are offset slightly so overlapping ones stay visible. Source: `circularity-pr/verify/figA_pr_jump_vs_variance.csv`.

What replaces it

- **For fidelity:** per-variable lane error (max |error| and fraction within tolerance), plus the *unexplained-variance fraction*, UVF = tr Cov(obs - exp) / tr Cov(obs). UVF splits exactly by lane group. Named lanes have UVF of at most 2.3e-7 everywhere. Scratch lanes reach at most 12.3% wherever the expected representation is non-empty. UVF is a proposal from this response and is not yet in the repository.
- **For the scientific question:** a task whose target is not read from the same residual, reported against a permutation null and trivial baselines. Examples are blind recovery against a perturbed schedule and decoding on held-out data.
- **PR itself:** report it only descriptively, divided by its value-frequency prediction. For the expected representation that ratio is 1.0 for every single-variable program; on the compiled model P05 and P06 depart from 1 because of their scratch lane.

4 Issue 2: numerical artifacts

2.1 P07, P08 and P10 at exactly 0.5

> *"P07, P08 and P10 jumping to exactly 0.5 looks like PR computed on near-zero noise."*

Verdict Agree **Status** Done

Answer. Correct. The original PR error was the mean of $|PR_{obs} - PR_{exp}|$ over 6 stages. At L1/Attn, L1/MLP and L2/Attn the observed PR was 1 and the expected PR was 0. The other three gaps were at most 1.1e-6. For P07 the mean was (1 + 1 + 1 + 1.7e-6) / 6 = 0.5000003. P11 had the same three noisy stages, which gave 3/12 = 0.25 of its 0.270. The original report had already localized the jump to these three stages, noted that total variance was not saved, and proposed recording it (original report, lines 447 and 510). We traced the noise to its source.

Property	Value (matched design, P07, P08, P10, P11)
Varying lane	`length_*_selector_width_attn_output`: the scratch attention output of `make_length`. It is the only computed-view lane with non-zero variance at these stages.
Exact value	1/7 for every probe (every probe has length 6)
Observed values	20,224 rows hold float32(1/7) = 0.1428571492; 74 rows hold 0.1428571343, exactly one ulp lower
Which rows	Rows 20,224-20,297: the final partial batch of the activation collector's 256-row loop (20,298 = 79 x 256 + 74; `geometry.py:268`; the same loop is at line 265 in 2dc9118)
Batch-size test	Batch sizes 1-83 give the low value; every tested size from 84 up (84-129, 200, 255, 256, 300) gives float32(1/7). The attention logits are bit-identical. The difference arises in the jitted forward pass (jax 0.11.2, XLA CPU). `jax.nn.softmax` alone does not reproduce it, so the exact operation was not isolated.
Variance and spectrum	8.07e-19, exactly rank 1, so PR = 1 exactly
Balanced design	No final batch is shorter than 84 rows, so the variance is exactly 0 and there is no artifact

Why PR came out at exactly 1. PR does not depend on scale. Scaling the noise by up to 1e18 leaves PR at 1.000000. A genuine signal (P07 L2/MLP) keeps PR 3.769 even when scaled down to a variance of 7.8e-25. Only magnitude separates noise from signal, and so does a batch-shape check (the noise disappears at batch sizes of 84 or more). For Tracr's O(1) activations the absolute floor is a workable numerical policy, but it masks the artifact rather than fixing it, and other models need a scale-relative form (Caveat 2 below).

After the floor (matched, computed view) the PR errors are 2.8e-7 (P07), 2.2e-7 (P08), 1.3e-7 (P10) and 0.020 (P11). **The root cause is still in the code:** the floor hides the artifact, but the 74-row final batch is still produced at `geometry.py:268`. Padding the last batch should remove it at the source: in this environment (jax 0.11.2, XLA CPU) the same tail rows, run inside a 256-row batch, give exactly float32(1/7). The pipeline has not yet been changed.

2.2 The 14 CKA availability mismatches

> *"The 14 CKA availability mismatches are likely the same thing."*

Verdict Agree, refined **Status** Done

Answer. 12 of the 14 were. With the floor, 2 remain, and they are real signal.

Program	Transition	Observed CKA (no floor)	Observed variance before	With floor 1e-12
P07, P08, P10, P11	L1/Attn -> L1/MLP	1.000 (noise vs itself)	8.07e-19 (noise)	Removed
P07, P08, P10, P11	L1/MLP -> L2/Attn	1.000 (noise vs itself)	8.07e-19 (noise)	Removed
P07, P08, P10, P11	L2/Attn -> L2/MLP	2e-6 to 4e-6	8.07e-19 (noise)	Removed
P05	L1/Attn -> L1/MLP	0.621	1.48e-2 (helper lane)	**Remains (real)**
P06	L1/Attn -> L1/MLP	0.773	1.13e-1 (helper lane)	**Remains (real)**

The remaining two come from the SelectorWidth helper lane. It varies one stage *before* the named count is written, while the expected representation, which zeros scratch lanes, is still empty. That variance is ten orders of magnitude above the floor. These mismatches come from how the benchmark is built (Section 3), not from numerics.

Correction: `research-validation.md` cites the L1/MLP variances (0.733 and 0.919) for these two. The variance that decides availability is at L1/Attn: 0.0148 and 0.113.

2.3 Save total variance, set a floor, rerun

> *"Fix: save total variance, set a variance floor, rerun the whole thing."*

Verdict Agree **Status** Done

Answer. Done, with two caveats on how much margin the floor has.

- **Saved.** Every stage table now stores observed and expected total variance (the sum of per-feature sample variances, ddof 1), the floor, and active flags with reasons (floor logic: `geometry.py:22, 351, 363, 391`). The comparison stage tables also keep raw floor-0 PR and CKA (`study.py:96-101`). Total variance was already in the original Step-3 `layer_metrics.csv`, where these stages show exactly 0 in the balanced design. The Step-4/5 stage tables are what is new.
- **Rerun.** All seven studies were rerun, and every manifest shows completed with no failures. A second comparison run gives numerically identical stage tables (its run folder is local and gitignored). The current code reproduces the matched stage tables of P05-P08, P10 and P11 to within 3.6e-15. Floor 0 reproduces the original Table 1 exactly.
- **Robust across tested floors.** We scanned 2,121 saved total variances. 197 are exactly 0. Only 12 lie in (0, 1e-12], all 8.07e-19 (the artifact). The smallest genuine value is 1.48e-2. Every floor from 1e-14 to 1e-8 gives identical PR errors, identical CKA pairs (24/54 computed, 54/54 full) and identical mismatches (2).

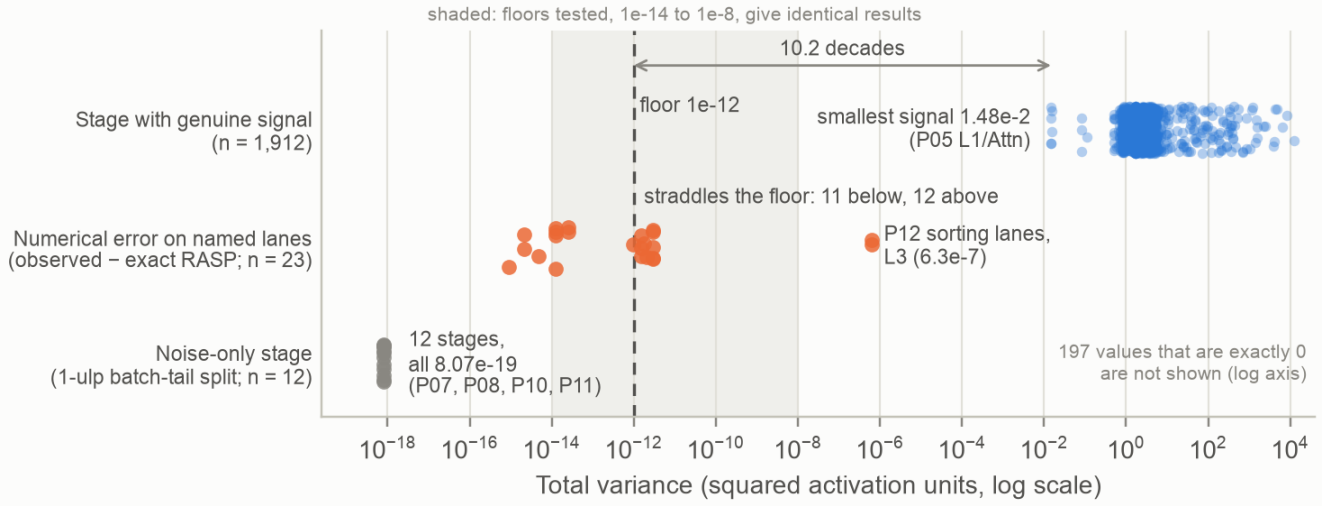


Figure 3. Every saved total variance, grouped by kind. The observed noise sits near 1e-18 and genuine signal at 1.5e-2 and above. Numerical error on written named lanes (float32 MLP and attention error, measured for this response) straddles the 1e-12 floor; P12's attention leak sits at 6.3e-7. Exact zeros (197) are not shown. Source: `numerics/verify/fig_floor_margin.csv`.

- **Caveat 1: the margin is small against known error.** The 16-order gap between the noise (8.07e-19) and the smallest genuine signal (1.48e-2) applies only to this one kind of noise. The pipeline's own numerical error on written named lanes (float32 MLP and attention error) has variance 1.7e-12 (P05), 1.5e-12 (P09) and 3.0e-12 (P11), at or above the floor; P12's attention leak (Section 5) reaches 6.3e-7. No conclusion changes, because that error rides on genuine signal, but a stage carrying only that error would pass the floor. A floor of 1e-8 would sit about 3.5 orders above the P05/P09/P11 error and about 6 below the smallest signal; no floor in the tested band would suppress an error as large as P12's leak.
- **Caveat 2: the floor is absolute.** It is in squared activation units. That is safe for Tracr's O(1) activations, but learned-model variances reach 1.27e4. Trained or compressed models need a scale-relative criterion, for example variance relative to float32 eps^2 times the squared norm.

Original claims withdrawn because of this artifact

Original Figs. 10-12: the claims that P07, P08 and P10 'rise from about 1e-7 to 0.5' and have computed-view errors 'about 0.5'. Those errors are now about 1e-7 in both designs and both views.

'14 vs 2 mismatches' (matched vs balanced) is now 2 vs 2.

P11's matched-versus-balanced comparison reverses: 0.270 > 0.207 becomes 0.020 < 0.207.

Original conclusion 5 (dependence on design and view) loses P07, P08 and P10 as evidence. It still rests on P05, P06, P09, P11 (now in the reverse direction) and P12.

5 Issue 3: P12 lane errors

3.1 Does the same 1/8 of observations fail everywhere?

> *"Exactly 87.5% in all 8 lanes means the same 1/8 of observations fail everywhere."*

Verdict Disagree **Status** Done

Answer. The row-level data show the opposite. Each of the 8 output lanes fails on exactly the 64 observations where it should read 1 (its *hot* observations). The 8 failing sets are pairwise disjoint and together cover all 512 observations. So every observation fails in exactly one lane, its own hot lane. The figure is 87.5% = 7/8 because the balanced probe design has 8 output classes with 64 targets each. Put plainly: 100% of observations have one output lane outside tolerance, and 0% have a wrong decoded answer.

Check (scale 1, L3/Attn, 512 probes x 8 lanes)	Result	Supports
Failing observations per lane	64 of 512 in each lane (87.5% pass)	Both
Lane j's failing set = observations where lane j is hot	True for 8 of 8 lanes	Disjoint hot lanes
Overlap between failing sets (28 pairs)	0 in every pair	Refutes 'same 1/8'
Intersection / union over the 8 lanes	0 / 512	Disjoint hot lanes
Hot entries (n = 512)	Error -8.52e-4 to -8.49e-4; all 512 outside tolerance (2e-4 at value 1)	Disjoint hot lanes
Cold entries (n = 3,584)	Error 0 to +3.6e-6; none outside tolerance (2,527 exactly 0)	Disjoint hot lanes
Matched design (20,298 observations)	Per-lane pass rates 87.21-87.87%; each lane's failure fraction equals its class frequency exactly	Disjoint hot lanes

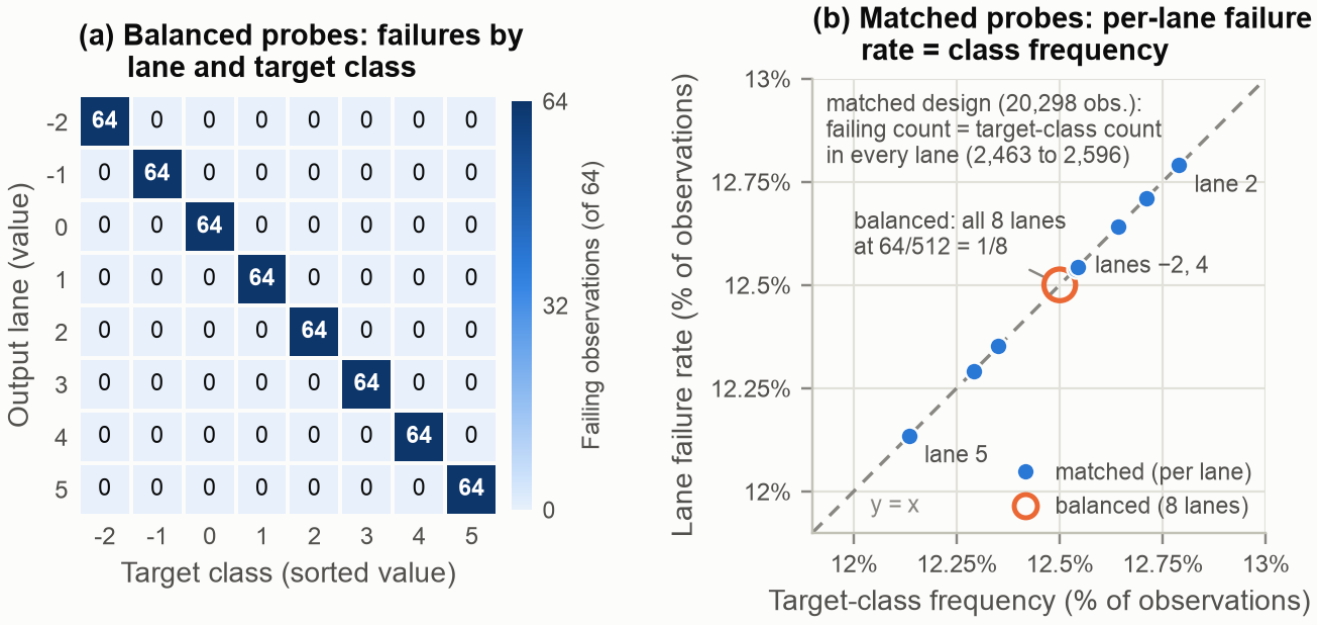


Figure 4. (a) Balanced probes: failing observations by output lane and target class. All 64 failures in each lane fall on the diagonal. (b) Matched probes, where classes are unbalanced: each lane's failure rate equals its class frequency, so the identical 87.5% under balanced probes is an artifact of class balance. Sources: `p12/fig2_lane_by_class_failures.csv`, `p12/verify/v_matched_vs_balanced_lane_failures.csv`.

This structure was reproduced on the original pipeline as well: Tracr's own forward pass on the original (2dc9118) probe file, not only the upgrade rerun. The original report's 98.41% P12 lane agreement is exactly 62/63, but that is what *either* hypothesis predicts (an average of eight lanes at 0.875 and 55 at 1.0), so it could not settle the question; only the row-level sets and the unequal matched-design rates do. The upgrade's decision log (`decisions.tsv`) and its committed change-report PDF already rejected the same-rows assumption. Only the upgrade report's generated

sentence '*same failing-probe set **False***' (`research-validation.md:143`) is a garbled boolean and should be reworded.

3.2 What causes the shortfall?

> *"Likely cause: softmax not saturating on ties or duplicates (??)"*

Verdict Partly agree **Status** Done

Answer. New analysis for this response; the upgrade ran the sweep but left the cause open (`research-validation.md:143`). The softmax is not saturating, but ties or duplicates in the input are not the cause. The output head puts 8.49e-4 of its attention on the BOS default slot (value 0), and it does so on every row. The match-vs-BOS logit gap is a constant 7.07 instead of Tracr's intended 50. The reviewer's instinct about duplicates is right in a roundabout way: the gap shrinks because of how `make_sort` *handles* duplicates.

- **Step 1.** P12 is `lib.make_sort(tokens, tokens, max_seq_len=6, min_key=1)` (`programs.py:161-168`). To break ties, `make_sort` makes keys unique as x + min_key * i / max_seq_len (`tracr/compiler/lib.py:217-218`). That gives 48 distinct key values.
- **Step 2.** The output Aggregate is categorical attention with softmax coldness 100. The logit is 100 for the match, 50 for BOS and 0 otherwise (`categorical_attn.py:144-149`). At the craft level the gap is 50, so the leak is about e^{-50} = 2e-22. Running Tracr's craft model directly gives zero shortfall on all 512 probes.
- **Step 3.** The assembled model uses one global key_size, the largest QK dimension of any head (`assemble.py:133`). It divides every logit by sqrt(key_size) (`attention.py:131`) and does not compensate in the weights. P12's layer-2 SelectorWidth head over 48 keys has a 50 x 49 QK matrix, so key_size = 50 and the output head's gap becomes 50 / sqrt(50) = 7.07.
- **Measured.** The layer-3 logits are identical on all 512 rows: match 14.142, BOS 7.071, others 0. The attention weights are match 0.999148 and BOS 8.486e-4.
- **Closed form.** With g = 100 / sqrt(50), Z = e^{g} + $e^{g/2}$ + 5, and the hot shortfall is ($e^{g/2}$ + 5 - (k - 1)) / Z, where k is the number of copies of the target value. This matches all 512 rows to within 5e-8. Duplicates make the error slightly *smaller* (k = 1: 8.522e-4; k = 6: 8.487e-4), because attention that leaks to another copy lands back in the hot lane. All 21 duplicate-free probes fail.

Program	Assembled key_size	Match-vs-BOS gap 50/sqrt(key_size)	Approx. leak e^{-gap}
P01-P03	1	50.0	no Aggregate head
P04, P09, A	8	17.7	2.1e-8
P05-P08, P10, B	10	15.8	1.4e-7 (where an Aggregate exists)
P11	14	13.4	1.6e-6
P12	**50**	**7.07**	**8.5e-4 (above the 2e-4 tolerance)**
P12 rewritten with `make_sort_unique` (counterfactual only; no tie-breaking, wrong on duplicate tokens)	10	15.8	1.4e-7

This explains why only P12 crosses the tolerance. Leak estimates are first order and cover Aggregate (BOS-default) heads only. The largest observed non-P12 lane error, 1.9e-6 on SelectorWidth/MLP lanes in P05, P06, P09 and B, has a different source and is also about 50x below the 1e-4 tolerance. Source: `p12/key_size_by_program.csv`, `p12/b_unique_counterfactual.py` (new analysis).

3.3 Sharpen attention and check the error shrinks

> *"Test: find the failing rows, sharpen attention, check the error shrinks."*

Verdict Agree **Status** Done

Answer. Done. The upgrade multiplied the QK logits by 1, 2, 4 and 8 before the softmax. The max lane error goes 8.52e-4, 7.15e-7, 1.35e-24, 0. Every tolerance failure is gone at scale 2. Token accuracy is 1.0 at every scale, including scale 1. New for this response: full-sequence sort accuracy is also 1.0, and sharpening only layer 3 reproduces the whole sweep, while sharpening only layers 1-2 changes nothing. The decay follows $e^{-s\ g/2}$ (the change in ln error from scale 1 to 2 is 7.08; predicted 7.07). The crossover scale for this tolerance is about 1.205 (closed form; scale 1.2 still fails, 1.25 passes). A scale of sqrt(50) = 7.07 would restore Tracr's intended coldness exactly.

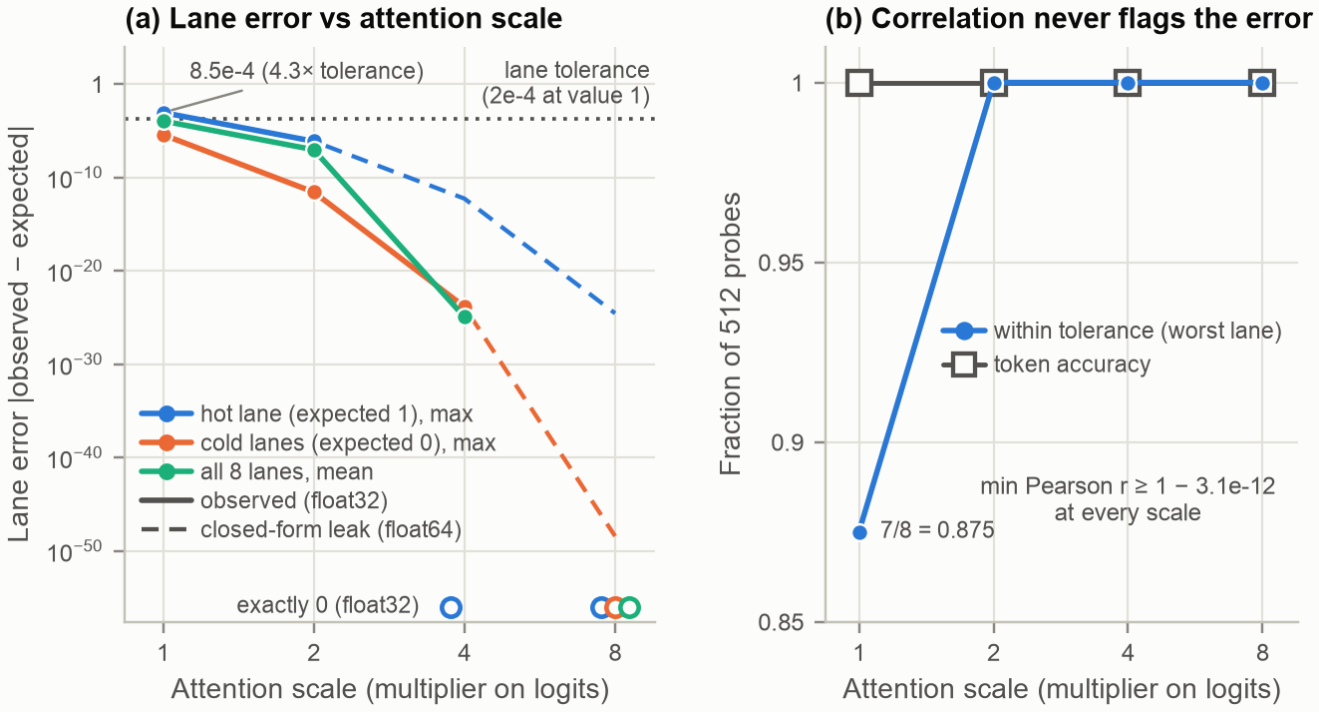


Figure 5. (a) Lane error against attention scale. Solid lines are measured; dashed lines are the closed-form softmax leak. At scale 4 the hot-lane shortfall drops below float32 resolution near 1.0, and the largest error left is a cold-lane leak. At scale 8 that leak underflows to exactly 0. (b) The fraction of probes within tolerance in the worst lane changes, but decoded accuracy and correlation do not (min Pearson r >= 1 - 3.1e-12 at every scale). Sources: `p12/fig1_sharpening.csv`, `context-repro/fig1_p12_correlation_vs_magnitude.csv`.

Effect on geometry, which bears on the headline critique. Sharpening to x8 changes P12's PR at L3/Attn by only +0.002 (full view) and -0.002 (computed view). CKA between the scale-1 and scale-8 representations is at least 0.9999997 wherever defined. So the lane error accounts for under 0.2% of P12's PR gap; as the original report already noted, its largest computed-view gap is at L2/Attn, before the output is written. Also, PR and CKA cannot see a magnitude error 4.3 times the tolerance.

6 Issue 4: the complexity analysis

4.1 Why does variable count equal depth?

> *"n = 12, and variable count equals depth for every program. Why is it this way?"*

Verdict Agree **Status** Explained

Answer. Because every one of the 12 programs is a single dependency chain. Tracr's depth counts variables (SOps) along the longest path from the inputs; Select nodes do not count (`craft_graph_to_model.py:59-62`). So depth can never exceed the variable count, and the two are equal exactly when all computed variables lie on one path. Rebuilding the 12 RASP graphs gives a maximum antichain, or DAG width, of 1 in every program. P01-P06 are single primitives. P07-P12 are linear pipelines in which each step reads only the previous computed variable plus raw tokens or indices.

ID	Computed-variable chain	Count	Depth	Layers
P01-P06	one primitive each (absolute, index_parity, increment_by_index, first_element, histogram, count_greater_than)	1	1	1
P07	length -> next_clamped_index -> next_value -> sum_with_next	4	4	3
P08	length -> previous_cyclic_index -> previous_value -> pairwise_sum	4	4	3
P09	shift_by(1) -> locally_increasing -> decrease_count -> check_increasing	4	4	3
P10	length -> next_index -> rotate_left	3	3	3
P11	length -> opp_idx -> opp_idx-1 -> reverse -> mirror_matches -> mismatch_count -> check_palindrome	7	7	6
P12	sequence_map (sort key) -> target_pos -> sorting	3	3	3

Layers = depth - 1 for P07, P08, P09 and P11. In each, Tracr merges an attention-only level with the MLP-only level after it (`craft_graph_to_model.py:170-180`). How this arose: the 12 programs appear to be a catalogue of standard sequence tasks covering the RASP primitives, with no topology criterion. That is our reading of `programs.py`; the repository does not record why they were chosen.

4.2 Two of three axes are collinear

> *"Two of three axes are collinear, so no conclusion is possible."*

Verdict Agree **Status** Withdrawn

Answer. Agreed, and it is slightly worse than stated. r(count, depth) = 1.000 and r(depth, layers) = 0.984. The centered three-axis matrix has rank 2 (singular values 10.37, 0.80 and about 0), and its second direction is just the layer-merge correction. There are only 4 distinct design points, and six programs sit at (1, 1, 1). The original Figure 13's y-values (matched, computed-view PR error) were also partly the numerical artifact: P07/P08/P10 at 0.5 fall to 1e-7 to 3e-7 under the floor, and P11 falls from 0.27 to 0.020. The original caption already noted that count and depth coincide and disclaimed a causal complexity claim (lines 480-490), but it did not explain why and kept the plot. We withdraw the complexity analysis of the original 12 programs. The upgraded notebook 05 still plots metrics against all three axes, and that plot should be removed or annotated.

4.3 Programs designed to vary depth and width

> *"Needs programs designed to vary depth and width."*

Verdict Agree **Status** Partly done

Answer. The upgrade adds a 3 x 3 family (`research.py:410-455`). It has b parallel branches, each a chain of d token-wise maps ((+-x + offset) mod 8), merged by a serial fold of b - 1 sums mod 8. This separates the axes structurally: requested d and b are orthogonal (r = 0), and DAG width = b. But the family cannot answer the reviewer's question:

- **The axes are still confounded.** Tracr depth = d + b - 1 and variable count = d x b + b - 1, so they remain correlated (r = 0.956), and layers = depth exactly in all 9 programs.
- **The branches carry no distinct information.** Every intermediate is a function of the current token alone. 39 of the 45 variables are bijections of the token (all 36 branch steps and the 3 final merges at b = 3), and the other 6 are 4-valued merges that all induce the same partition {t, t+4}. At d = 2, all branches compute the same function.
- **So geometry is a closed-form function of what has been written.** Each stage's PR, total variance and CKA depend only on how many token copies and merges have been written. A serial depth-3 chain and three parallel maps written in one layer give the same PR (9.718795) and CKA. The closed form (new analysis) reproduces all 63 stages to within 1e-13 (Figure 6b).
- **The upgrade measured little on the family.** It recorded only PR and total variance: no CKA, gaps, lane checks or recovery, and no regression.

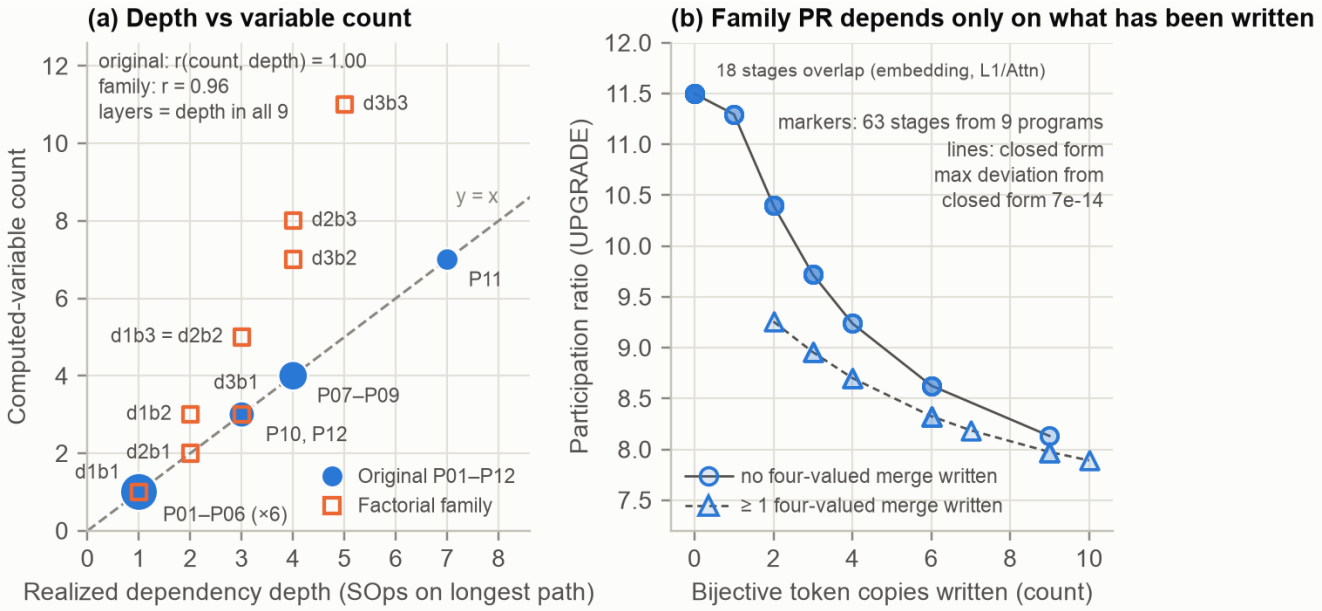


Figure 6. (a) Computed-variable count against realized dependency depth. The original 12 programs all lie on the diagonal, at 4 distinct points. The factorial family moves off it but still rises with depth (r = 0.96). (b) In the family, stage PR lies exactly on a closed-form curve in the number of token copies written, with or without the 4-valued merge, whatever the depth or branching. Sources: `complexity/fig1_depth_vs_variable_count.csv`, `complexity/verify/fig2_alt_family_pr_closed_form.csv`.

For description only (n = 9, deterministic metrics, new analysis): final PR falls 0.55 per step of requested chain length d and 1.11 per branch (R^2 0.91). On realized depth and count the two coefficients cannot be separated (-0.51 +- 0.53 and -0.13 +- 0.20). Output cardinality also varies with b (4 values at b = 2, 8 otherwise), so the branch coefficient is not a pure width effect.

A design that could answer the question

- Hold realized depth fixed while varying the number of branches, using a balanced merge tree with bijective padding for shallower cells.
- Give intermediates distinct information, for example attention hops that read different positions, such as combining `shift_by(+1)` and `shift_by(-1)`.
- Cross value cardinality (2, 4, 8) as its own factor, because PR tracks it.
- Run 3-5 random instances per cell, and pre-specify the analysis (ANOVA).

- Run CKA, lane checks and blind recovery on the family, and add programs where count > depth.

7 Issue 5: interventions

5.1 A patches the output directly

> *"A patches the output directly, so it's trivially correct."*

Verdict Agree **Status** Conceded

Answer. Correct. A's patched variable is the program output. It is written by A's only active block, the layer-1 attention, and the next step is the unembedding. So the 512 of 512 donor matches show only that the readout reads the lanes we overwrote. The original called A a 'direct-readout control' (original report, line 305), but still counted it in the 'All 1,280 conditions agree' headline. Of the 1,280 conditions, 640 are no-op controls, 512 are A readout overwrites and 128 are B donors. We withdraw A as causal evidence.

A further concession. The upgrade's geometry-selected patches repeat this design in 7 of 12 programs (P01-P06 and P10). There, the selected stage's variable is the program output, with no non-zero block before the unembedding. Every PCA-patch success in the upgrade's stored pairs happens at one of these sites (point 5.4).

5.2 B is one dependency with outcome-selected donors

> *"B is one dependency, with donors picked to flip the answer."*

Verdict Agree **Status** Partly done

Answer. Correct. The donor is the first probe at the same position with a different output class (`alignment.py:199-202`). So all 128 donor counts cross the count > 1 threshold. There are only 12 distinct donor sequences and one downstream MLP. The original disclosed the selection (line 330). The upgrade instead samples 16 (recipient, donor, position) triples per program without looking at outcomes, across all 12 programs. But only 15 of the 192 stored pairs actually test a downstream dependency (14 in P08, 1 in P11). The same 16 index triples are also reused for every program.

5.3 No connection to the Step 3 geometry

> *"Neither connects to the geometry from Step 3."*

Verdict Agree **Status** Partly done

Answer. True of the original, which took its lanes from the compiler. The upgrade picks the patch stage as the calibration transition with the largest 1 - adjacent CKA (`research.py:108-109, 245`). It saves that choice before the schedule is loaded, then patches donor-minus-recipient differences projected onto the top 1 or top 3 calibration PCs. Three qualifications apply. PR is not used. The geometry is refitted on a new calibration split rather than reusing Step 3. And choosing the stage is nearly trivial in compiled models: transitions that write no compiler variable change by 0 or almost 0 (at most 0.024, from helper-lane writes), well below every selected write (at least 0.051). So the 12 of 12 hits on write transitions are weak evidence; six programs have only 2 transitions, and for the six multi-layer programs alone a chance 6 of 6 has probability 0.043.

5.4 Patch along PCA directions or high-CKA transitions

> *"Better: patch along PCA directions or high-CKA transitions and check they map to variables."*

Verdict Agree **Status** Partly done

Answer. PCA-direction patching was run, and the results are weak. Patching at high-CKA transitions was not run. Over all 192 pairs, the match with the symbolic counterfactual is 0.479 for self (do nothing), 0.464 for random rank 1, 0.521 for PCA rank 1, 0.422 for random rank 3, 0.589 for PCA rank 3 and 0.948 for the compiler-lane oracle. Self scores 0.479 because in 92 of the 192 pairs the substitution leaves the output unchanged; another 10 (all P12) have undefined counterfactuals, so only 90 pairs are informative. Splitting by patch site shows where PCA succeeds:

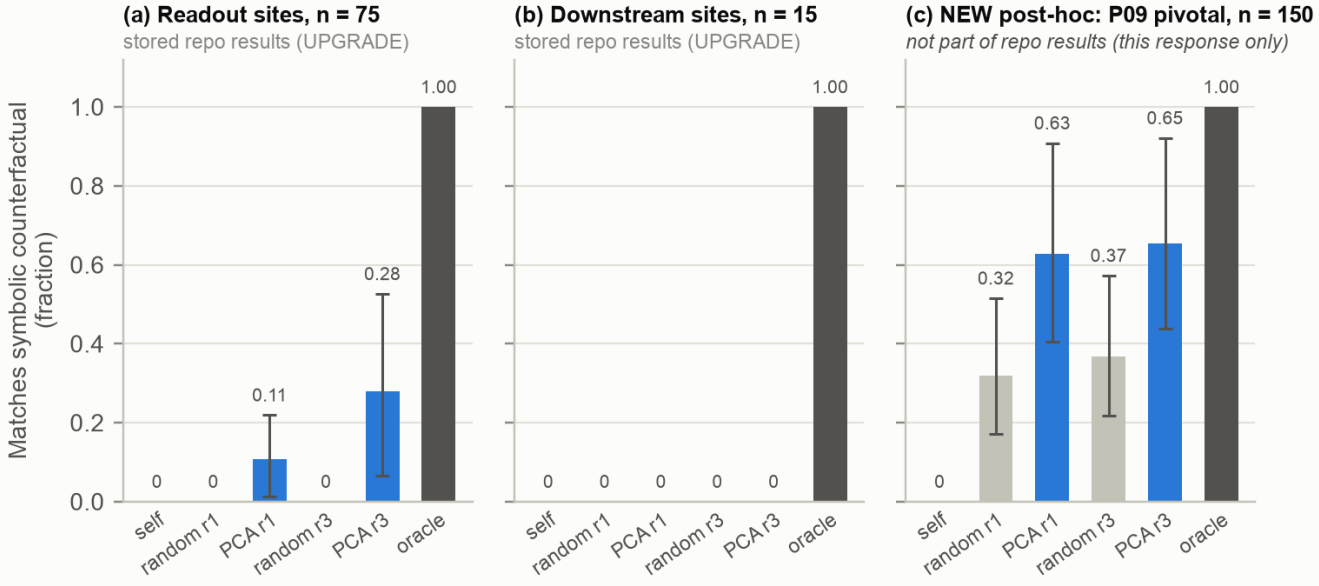


Figure 7. Rate at which the patched output matches the symbolic counterfactual, on pairs where the counterfactual changes the output. (a) Upgrade readout sites (P01-P06, P10). (b) Upgrade downstream sites (P07, P08, P09, P11, P12). Error bars are program-cluster bootstrap 95% CIs. (c) New post-hoc test, not in the repository's results: 150 outcome-pivotal P09 triples, using the upgrade's own stage and bases. Error bars are recipient-cluster CIs. Source: `interventions/verify/fig_skeptic_readout_vs_downstream.csv`.

- **Readout sites (75 informative pairs):** PCA rank 3 = 0.28 (program-cluster CI 0.07-0.53); PCA rank 1 = 0.11; random and self = 0; oracle = 1.0. All successes come from P01, P05 and P06. Those are the programs where the top three PCs capture most of the variable's own variance (0.74, 0.75 and 0.53), compared with 0.13 (P03), 0.39 (P04) and 0.02 (P10), the other readout programs with informative pairs. The random arm is one basis per program (P04-P06 share it), not a distribution, and PCA rank 3 also gives 5 wrong changes on informative pairs.
- **Downstream sites (15 informative pairs):** PCA = 0 of 15; oracle = 15 of 15. Across all 80 stored pairs at downstream sites, PCA patches never changed the output.
- **'Map to variables' was never tested independently.** At the selected stage exactly one compiler variable is first written in every program, so the decoder never had to choose. The 'mapped' variable comes from the schedule. An unrestricted decoder picks the stage's new variable in only 3 of the 6 multi-variable programs.
- **New post-hoc downstream test.** P09 has 2,243 outcome-pivotal triples in its test split; the upgrade's 16 triples hit none of them. On 150 of these triples, PCA rank 3 reproduces the counterfactual through three downstream blocks in 98/150 = 0.65 (recipient-cluster CI 0.44-0.92), and PCA rank 1 in 0.63. Random rank 3 manages 0.37 (CI 0.22-0.57), self 0 and the oracle 1.0; the PCA and random CIs overlap. It also works in one direction only: repairs (False to True) succeed 94/94, versus 51-59% for random, while breaks (True to False) succeed 4/56. On P11 pivotal triples, PCA scores 0/150. This is weak, direction-dependent evidence that geometry-guided patches *can* carry a variable downstream.
- **High-CKA transitions: not done.** CKA picks only one (lowest-CKA) transition per program. Nothing compares stages, derives directions from CKA, or patches the trained models.

Next: patch at every transition, ranked by 1 - CKA, with the same pairs and arms. Sample outcome-pivotal pairs. Use learned (DAS-style) directions. Repeat on trained models.

8 Issue 6: transfer limits

6.1-6.2 No superposition, no erasure, never overwritten

> *"Tracr has no superposition and no erasure, and its lanes are axis-aligned. Variables are never overwritten, so compression and lifetime effects can't appear."*

Verdict Agree **Status** Open

Answer. Agreed completely, and confirmed in the stored activations. 'Open' refers to erasure and lifetimes; the axis-aligned and superposition parts are partly covered by N1 and N2 (table below). The original report already said it was 'not a live-variable benchmark' (original report, line 285). Across 52 variables in 12 programs:

- Each variable's lanes are disjoint from every other variable's.
- Every variable lane is exactly 0.0 before its first write and changes by exactly 0.0 afterwards.
- All 9 SelectorWidth scratch lanes stay non-zero until the final stage.
- At the final stage, 32-90% of the residual width (mean 61%) is still occupied by non-output intermediates and inputs.
- The expected representation builds in the same assumption: each encoding is inserted 'at its scheduled first write and retain[ed] thereafter' (original report, line 283); `alignment.py:72` sets those lanes once and never clears them.

Program	Width	Max |lane| before write	Max |change| after write	Width held by intermediates and inputs at the end
P01	22	0.0	0.0	0.36
P02	19	0.0	0.0	0.74
P03	30	0.0	0.0	0.47
P04	25	0.0	0.0	0.56
P05	25	0.0	0.0	0.32
P06	25	0.0	0.0	0.32
P07	55	0.0	0.0	0.65
P08	54	0.0	0.0	0.65
P09	37	0.0	0.0	0.84
P10	39	0.0	0.0	0.69
P11	69	0.0	0.0	0.90
P12	81	0.0	0.0	0.85

Upgrade test activations, recomputed for this response (`transfer/lifetime_*.csv`). Scratch lanes and unused input lanes are excluded from the last column, so its values are lower bounds.

How the upgrade handles each limit

Limit	Upgrade experiment	Covered?
Axis-aligned lanes	Random orthogonal rotation (Section 9)	Yes, as a control
Superposition	Untrained linear compression (Section 10)	Partly: lanes overlap, but nothing is learned
Erasure / lifetimes	None	No

Proposed lifetime tests

- **Erase-and-reuse pass.** After a variable's last reader, add an MLP that writes -x on its lanes (hidden units ReLU(x) and ReLU(-x) with output weights -1 and +1 write -x exactly), then reuse those lanes. This gives ground-truth erase and overwrite events. Do drops in variance, PR or CKA flag erasures? Can a blind detector tell an erase from a write?
- **Live-only expected representation.** Keep only live variables in the expected representation and rerun the PR/CKA gaps.
- **Combined stress test.** Apply erasure together with rotation and learned compression.

- **Lifetime curves in trained models.** Measure, layer by layer, how well a linear probe decodes each RASP variable.
- **Deep chains under a fixed width budget,** where dead intermediates must be cleared.

9 Next step 1: random orthogonal rotation

N1 PR and CKA should hold while lane checks break

> *"Random orthogonal rotation: PR and CKA should hold while lane checks break."*

Verdict Agree **Status** Done

Answer. Done, and the result is as predicted. The upgrade rotates all 12 compiled models by random orthogonal matrices (seeds 42-44), both after the fact and as executed models. In the executed models, embeddings, every read and write weight, biases and the unembedding are transformed (`models.py:127-149`).

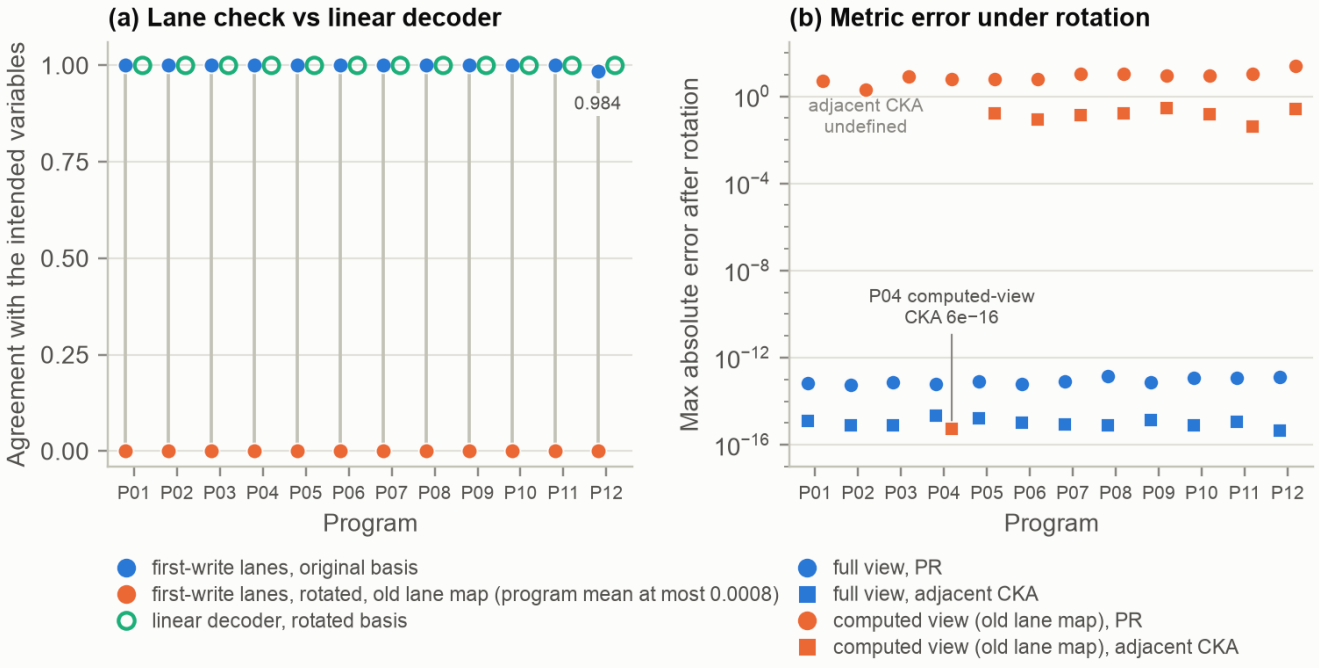


Figure 8. (a) First-write lane agreement drops from about 1 to about 0.0003 after rotation (old lane map), while a linear decoder fitted on rotated calibration data still reads every variable. (b) Maximum absolute error under rotation, log scale. Full-residual PR and CKA are unchanged to rounding error. Computed-view metrics, which select lanes by label, change by up to 24 (PR) and 0.29 (CKA). Source: `transfer/fig2_rotation_invariance_vs_lanes.csv` (all values recomputed for this response from the upgrade's stored activations; the upgrade itself stored only whole-residual PR and all-pair CKA errors).

- **PR and CKA hold.** Whole-residual PR error is at most 1.4e-13 and all-pair CKA error at most 2.2e-15. The executed rotated models give identical outputs on 100% of tokens; executed-state PR error, recomputed for this response, is at most 2.2e-5 (float32).
- **Lane checks break.** The repository's first-write lane check with the old lane map, recomputed for this response on the upgrade's validation test split, falls from 0.996 to 0.0003 (269 lanes x 3 seeds; no lane above 0.026). The upgrade's stored `canonical_lane_agreement` column is a whole-residual element-wise comparison, not this check.
- **The information survives.** A linear decoder fitted on rotated data reads every variable (1.000), so only the axis-aligned readout breaks. The decoder is more permissive than the lane check (1.000 on P12 even unrotated, where lanes give 0.984), and its invariance is guaranteed by construction.
- **The computed view breaks too.** It is defined by lane labels. Under rotation its largest PR change per program ranges from 2.0 (P02) to 23.8 (P12), and 28 of 66 stages switch from undefined to defined. Blind-detector F1 is unchanged.

Honest framing. That PR, linear CKA, energy and affine decoders are unchanged by rotation is guaranteed mathematically. The experiment checks the implementation; it does not show that PR or CKA reveal computation. The

informative part is the *non-orthogonal* control. Rescaling each axis by geomspace(0.1, 10) changes PR by -2% to -73% (mean -49%) and adjacent CKA by up to 0.22, while uniform scaling changes nothing (the adjacent-CKA change and the uniform-scaling control were computed for this response). This is one fixed scale vector spanning a 100x range, assigned by lane index, so the size of the effect depends on that choice, which the upgrade did not vary. Per-feature scales, such as LayerNorm gains, could move these metrics with no change in computation; how much depends on how unequal the scales are.

10 Next step 2: compression and superposition

N2 Compress the residual, then retest

> *"Compress the residual to induce superposition, then retest."*

Verdict Agree **Status** Partly done

Answer. Compression was run, but it is an untrained linear projection, not learned superposition. The retest is a negative result for the detectors.

What was done. The basis comes from an uncentered SVD of calibration residuals, or is random and orthonormal. Its rank is ceil(f x width) for f = 0.25, 0.5, 0.75 and 1. Embeddings, read and write weights, biases and the unembedding are projected onto it. Attention and MLP hidden widths are unchanged, and nothing is retrained. The Tracr paper (Lindner et al., 2023, Section 5) uses the same read/write projection but *trains* it by gradient descent with every compiled weight frozen. For example, it compresses frac_prevs from 14 to 6 dimensions without hurting performance. That training step is what is missing here.

Condition	Width fraction	Token acc. (all 12)	Seq. acc. (all 12)	Token acc. (held-out)	Mean axis overlap	Same-stage CKA (median)	PR change
PCA f = 0.25	0.267	0.413	0.180	0.376	0.247	0.846	-29.6%
PCA f = 0.50	0.511	0.616	0.278	0.643	0.122	0.978	-8.4%
PCA f = 0.75	0.764	0.721	0.631	0.751	0.045	0.99994	-3.1%
PCA f = 1.00	1.000	1.000	1.000	1.000	~1e-16	1.000	0%
Random f = 0.50	0.511	0.370	0.075	0.376	0.139	0.778	-25.0%

Accuracy columns are macro means at the final stage; axis overlap is a mean over programs; same-stage CKA and PR change are pooled over all 54 post-embedding stages (at the final stage alone the CKA median is 0.659/0.896/0.987/1.000/0.698 and the PR change -51%/-19%/-6%/0/-40%). Held-out = P07-P12. Per program, 34 of 48 compressed models beat their own majority-class baseline, 3 tie with it (P11, constant output) and 11 fall below it. Six compute perfectly: P01, P07 and P10 at f = 0.75, and P02 at 0.25, 0.5 and 0.75. P11 collapses to a constant predictor at every PCA f < 1 (the random-basis model scores 0.878, below its baseline but not constant). The maximum |cos| between projected axes reaches 1.0 at every PCA f < 1, but this is measured on the basis, not the data: P10 at f = 0.75 reaches 1.0 yet compresses losslessly, and axes that project to zero are excluded. Source: `results/validation/reference/p*/compression.csv`.

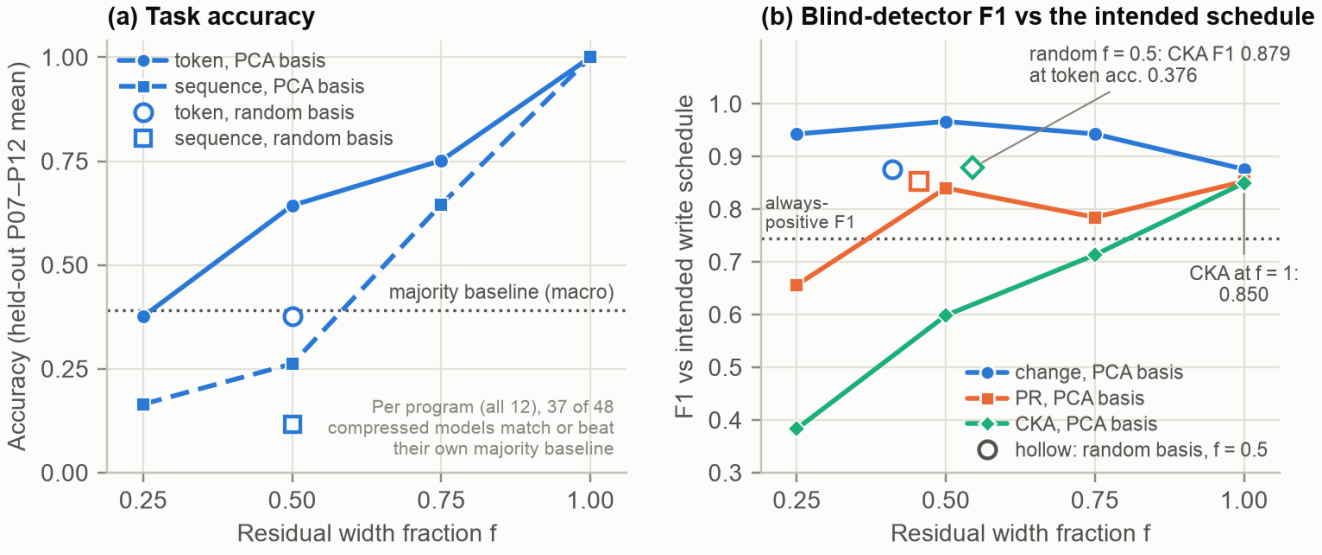


Figure 9. (a) Held-out task accuracy against residual width fraction. (b) Blind-detector F1 against the original compiler schedule. Random-basis compression (hollow markers) breaks the task, yet detector F1 matches the intact model for change (0.876 vs 0.876) and PR (0.852 vs 0.853) and exceeds it for CKA (0.879 vs 0.850). The x-axis is nominal f; realized widths are 0.267, 0.511 and 0.764. Source: `transfer/fig1_compression_accuracy_vs_f1.csv`.

- **Random basis, f = 0.5.** Held-out accuracy falls to 0.376, but CKA-detector F1 rises from 0.850 to 0.879. Within this condition, CKA F1 and token accuracy are unrelated across all 12 programs (Spearman rho = -0.05, n = 12). For example, P08's CKA F1 goes from 0.857 to 1.000 while its token accuracy is 0.087.
- **PCA basis.** CKA F1 falls with accuracy (0.850, 0.713, 0.598, 0.383 at accuracy 1.0, 0.751, 0.643, 0.376), and PR F1 falls overall but not monotonically (0.853, 0.785, 0.840, 0.656). The drop is lost recall: writes become too small for the PR/CKA thresholds (CKA recall 0.25-0.57), not detection of broken computation. Change-detector F1 rises to 0.94-0.97.
- **Not done.** The expected-representation comparison (PR/CKA gaps and lane checks against RASP) was never run on compressed models. It could run now on the six models that still compute correctly, but four of them (P01, P07, P10 and P02 at f = 0.75) are lossless or nearly so. Only P02 at f = 0.25 and 0.5, a position-only program, computes correctly with genuinely lossy, overlapping lanes.

Next: use the existing JAX code to train only the projection, on output loss plus per-layer loss, as in Tracr Section 5. Then rerun the full benchmark and blind recovery on models that compute correctly *in* superposition.

11 Next step 3: trained models

N3 Train or fine-tune on the same programs

> *"Train or fine-tune models on the same programs and compare to compiled kernels."*

Verdict Agree **Status** Partly done

Answer. The upgrade trained 36 transformers from random initialization: 12 programs x seeds 42, 43 and 44. No compiled weights were fine-tuned. Each model has width 32, 2 layers, 2 heads, MLP width 64, no LayerNorm and non-causal attention. Training ran 600 Adam steps (batch 128, learning rate 3e-3) and kept the final checkpoint. The data are 3,383 unique length-6 sequences (1.29% of 8^6), split 2,029 / 677 / 677 by whole sequence. For this response we retrained all 36 from scratch: the checkpoints are byte-identical (105 s).

Program	Majority baseline	Learned token acc. (3 seeds)	Minority-class recall	Reading
P01-P04, P10, P12	0.12-0.50	1.000	1.000	Learned
P05, P06	0.39, 0.30	0.9985	~1.0	Learned; every seed fails only the constant [-2]x6
P07, P08	0.11	0.9998-0.9999	1.000	Learned
P09 check_increasing	0.928	0.989 (0.985-0.992)	0.968; balanced acc. 0.98	Learned; the small margin is class imbalance
P11 check_palindrome	0.939	0.954 (0.952-0.958)	0.40; balanced acc. 0.70	**Not reliably learned**; validation loss rises late for 2 seeds

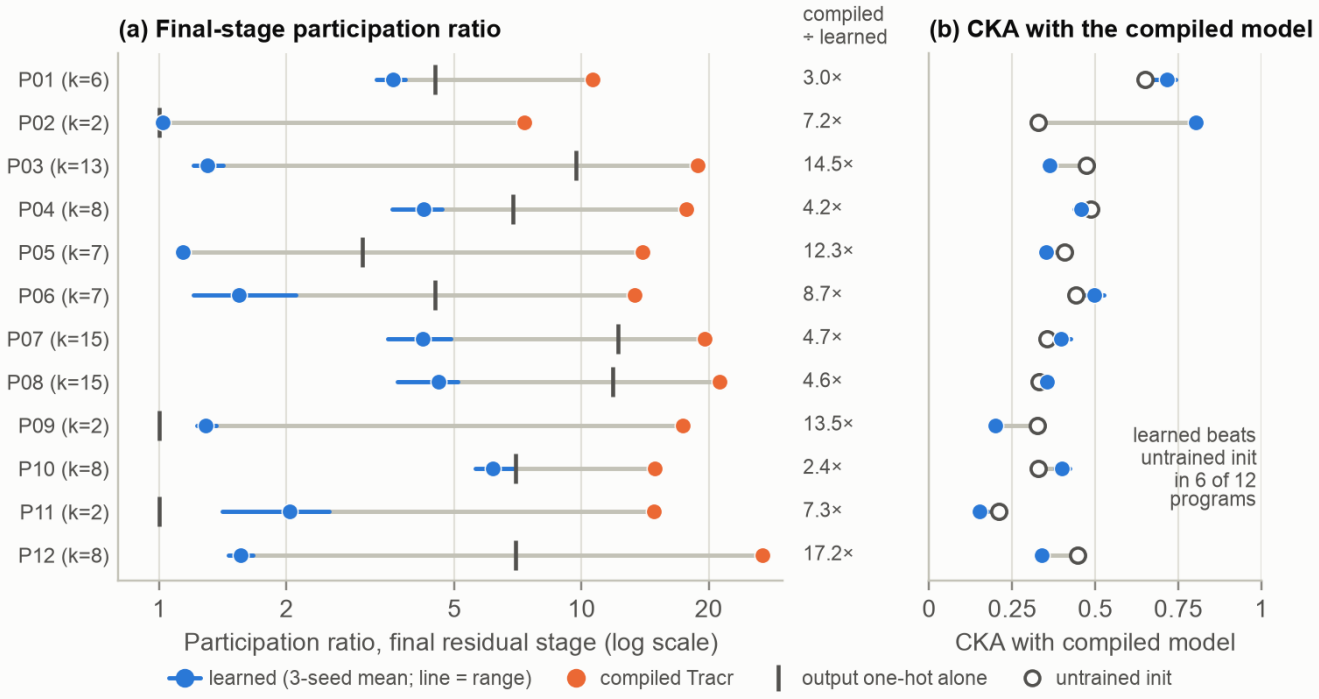


Figure 10. (a) Final-stage PR of learned models (3 seeds, whiskers = range) and compiled models; ticks mark the PR of the output one-hot alone. (b) Final-stage CKA with the compiled model, for the trained model and for its untrained same-seed initialization. Source: `learning-blind/fig2_learned_vs_compiled_final_stage.csv`.

- **PR.** Learned final PR is 1.02-6.16 against compiled 7.32-26.84, 2.4-17x lower. Learned PR *falls* from embedding to output in 12 of 12 programs, while compiled PR *rises* in 10 of 12. P03 has 13 classes but learned PR 1.30. PC1 correlates |r| >= 0.91 with the output in 7 programs, so learned models use low-dimensional, often ordinal codes. Confounds: compiled PR is measured on a residual that still holds the inputs and all intermediates (embedding PR 11.7), whereas trained models erase them; learned final-stage residual variance is about 10-3,500x the compiled

variance (program means; 7-4,900x per seed; no LayerNorm or weight decay); and only one architecture was trained.

- **CKA.** Final-stage learned-vs-compiled CKA is 0.15-0.80. But an untrained same-seed initialization scores 0.21-0.65, and the trained model beats it in only 6 of 12 programs. The reason is that the compiled final residual still contains the inputs (CKA with its own embedding is 0.60-0.91), while trained models erase them (0.006-0.46). This is the no-erasure limit again. P02's 0.80 is mostly the shared output (CKA with the output one-hot is 0.98).
- **Reading.** Absolute PR and CKA reflect the *encoding* (one-hot, append-only) more than the function. Compiled values cannot serve as references for trained models. Stages are paired only by relative depth (`study.py:218`); only the embedding and final stages are aligned in meaning.

Still open: fine-tuning compiled weights, probing trained models variable by variable, a seed-to-seed CKA ceiling, and length generalization.

12 Next step 4: blind recovery of the write schedule

N4 Can the metrics recover the write schedule from activations alone?

> *"Blind recovery: can the metrics recover the write schedule from activations alone?"*

Verdict Agree **Status** Open

Answer. No. On compiled models, the detectors recover one yes/no flag per sublayer, and that flag can be read from the weights. They recover nothing about which variable is written, how many, in which lanes, or for how long. On trained models they cannot be scored, and they fire almost everywhere.

Setup (upgrade). Three detectors run on consecutive stages over 576 test observations each: change (relative energy > 1e-6), PR (|delta PR| > 0.05) and CKA (1 - CKA > 0.01). A transition counts as true if at least one named variable is first written there. Predictions are hashed before the schedule is loaded. P01-P06 form the calibration group and P07-P12 are held out.

Detector	Held-out macro F1	Pooled exact p	Note
change	0.876	6.7e-5	Identical to a weights-only 'output projection non-zero' rule at 54/54 transitions
PR	0.853	3.4e-5	Same as change except at the 8 transitions involving the probe-constant `length` variable
CKA	0.850	5.7e-6	Only 7 other calls: 4 correct helper rejections, 3 missed named writes
Every-MLP rule (no activations)	0.805	1.2e-4	Structural baseline
Always positive	0.745	-	Writes are frequent (prevalence 0.60)

17 of 18 held-out program x detector cells beat always-positive. Per-program $p < 0.05$ holds in only 1 of 18 cells (stored Monte Carlo) or 2 of 18 (exact). In 5 of the 6 held-out programs even a perfect detector could not reach $p < 0.05$, because the minimum p is $1/C(n,t) = 0.05$-0.067. In the calibration group (P01-P06, 2 transitions each), 13 of 18 cells beat always-positive and 5 only tie it; overall, 6 of 36 program x detector cells are at or below the baseline. Sources: `results/validation/reference/p*/write-recovery-scores.csv`; `learning-blind/recovery_*.csv`.

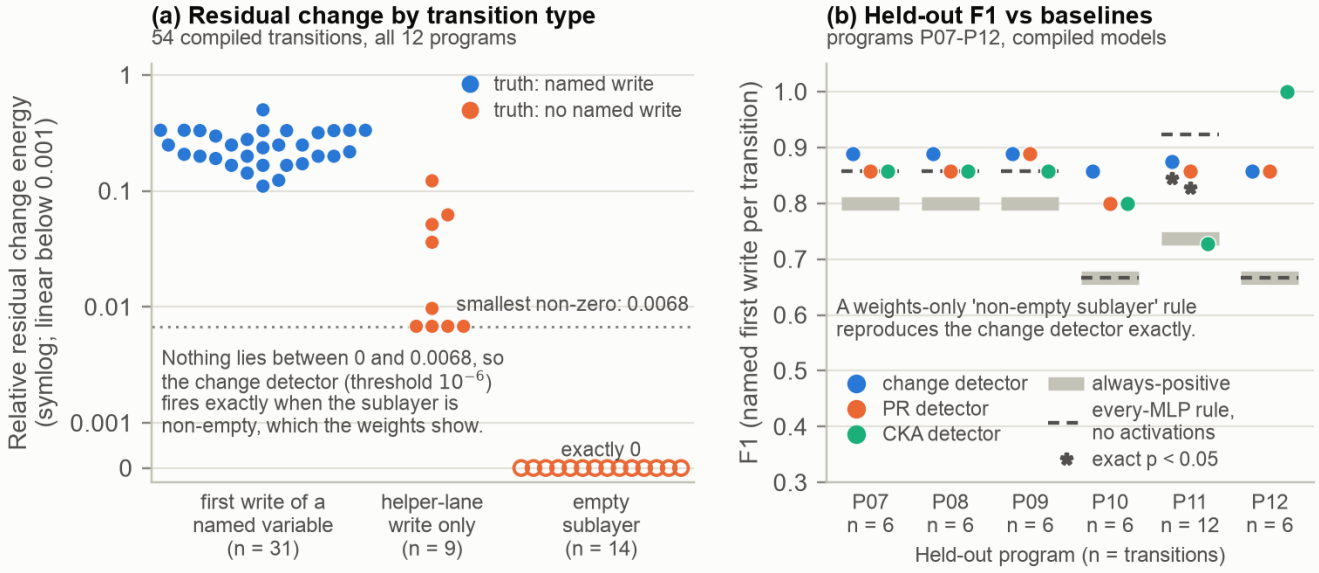


Figure 11. (a) Relative residual change at all 54 compiled transitions. Empty sublayers change by exactly 0 and every other sublayer by at least 0.0068, so the change detector reduces to 'is this sublayer non-empty?'. (b) Held-out F1 for each detector against the always-positive bar and the activation-free every-MLP rule. Sources: `learning-blind/verify/fig_transition_energy_by_category.csv`, `learning-blind/fig1_heldout_write_recovery_f1.csv`.

- **Why it is nearly trivial.** There are 31 first writes (relative energy 0.110-0.500), 14 transitions with exactly zero change (empty sublayers) and 9 writes to helper lanes only (0.0068-0.122). PR and CKA are centered, so they cannot see a variable that is constant across probes, such as `length`. P12's perfect CKA F1 rests on a helper write sitting 0.0011 below the threshold.
- **Protocol.** The thresholds are hard-coded (`research.py:118-119`), not fitted on P01-P06; the calibration group is only a label. Thresholds that are optimal on calibration would give held-out change F1 anywhere from 0.34 to 0.99. The freeze is real in code, because `infer_writes` never receives the schedule. But it is not a preregistration: the compiled schedules were public before the thresholds were set.
- **Trained models.** The upgrade did not run the detectors on them. Running the frozen detectors on all 36 for this response flags 144/144 (change), 141/144 (PR) and 138/144 (CKA) transitions, which is effectively always-positive.

What a real test needs: known schedules that do not show up as empty sublayers, for example SIIT models, compiled models fine-tuned end to end, or compiled models with inserted non-empty no-op sublayers and delayed writes. (Rotation and learned compression change the lanes but leave empty sublayers exactly empty.) It also needs threshold-free scoring (AUROC) and variable-level targets (which variable, which subspace) via DAS or probes.

13 Positioning against InterpBench

N5 InterpBench, the closest prior work

> *"Position against InterpBench (Gupta et al., 2024), the closest prior work."*

Verdict Agree **Status** Partly done

Answer. The original did not cite InterpBench. The upgrade adds a paragraph (`research-validation.md:228-230`) and says no InterpBench model was run. Below are a verified description, a comparison, and a concrete experiment.

What InterpBench is. Gupta, Arcuschin, Kwa and Garriga-Alonso, *InterpBench: Semi-Synthetic Transformers for Evaluating Mechanistic Interpretability Techniques*, NeurIPS 2024 Datasets and Benchmarks (arXiv:2407.14494). It releases 86 models: 85 derived from Tracr-compiled RASP programs and 1 trained on a simplified IOI circuit. They are trained with Strict Interchange Intervention Training (SIIT). SIIT is IIT plus a strictness loss that makes components outside the circuit irrelevant on the task. The paper evaluates ACDC, node and edge Subnetwork Probing, EAP and EAP-IG by edge-level AUROC, on 16 main Tracr models plus IOI. The models and their ground-truth edges are on Hugging Face.

	InterpBench	RaspFormer
Question	Do circuit-discovery methods recover the known circuit?	Do PR, CKA and PCA, and blind detectors, reflect the known computation?
Models	86 SIIT models (85 Tracr-derived, 1 IOI)	12 compiled programs + A/B + 9 family variants; 36 trained models
Weights	Trained, argued to be closer to natural models	Exact compiled weights; trained models have no known circuit
Ground truth	Circuit edges (`edges.pkl`)	Per-lane RASP values and write times
Metric	Edge AUROC; Wilcoxon-Mann-Whitney; Vargha-Delaney A12	Lane error, PR/CKA gaps, transition F1 against nulls and trivial baselines
Unique controls	IIT vs SIIT strictness; realism comparisons	Variance floor with raw metrics kept; full vs computed views; matched vs balanced; rotation; executed compression
Overlapping tasks	Case 41 absolute, 7 token counts, 44 count-greater; case 2 reverse	P01, P05, P06; reverse appears only inside P11

Positioning. The two projects are complementary. InterpBench supplies what RaspFormer lacks: realistic trained weights *with* internal ground truth, at scale. RaspFormer supplies value-level ground truth (which residual coordinates hold which RASP value, and which variable each sublayer writes) that an edge list does not contain. It also brings an explicit numerical policy, controls on the probe design and basis, and a blind-detection protocol (frozen predictions, count-matched nulls, trivial baselines), although on compiled models that protocol currently reduces to detecting non-empty sublayers.

Integration experiment (designed, not run)

- Take the 3 direct task overlaps plus about 20 other Tracr-derived cases. For each, compare the compiled Tracr model with its SIIT model. The 3 overlaps are among the 69 models added after InterpBench's evaluation, so ACDC/EAP baselines must be run on them with the InterpBench code; draw the other cases from the 16 main evaluated models, where published AUROCs exist.
- Cache the residual stream at every block boundary on class-balanced probes.
- Run the detectors with **threshold-free ranking** (AUROC) against `edges.pkl`, or with thresholds calibrated on held-out models. The fixed thresholds were set on sparse compiled models and would probably degenerate to always-positive on dense SIIT models.
- Compare PR and CKA trajectories across the two weight types. Rank nodes by the effect of top-k PCA-direction patches and compute AUROC next to ACDC and EAP.

- **Falsifiable prediction:** if the geometric detectors only exploit Tracr's empty sublayers and axis-aligned lanes, their AUROC will fall to chance on SIIT models.
- **Needs:** torch, transformer_lens, huggingface_hub and the InterpBench circuit-discovery code, which are not in the current environment. Lane checks will not transfer, because SIIT does not preserve residual coordinates.

14 The parts you liked: what survived

Item	Original	Upgrade	Status
Reproducibility hygiene	8 tests; 2 committed manifests; R4/R5 raw tables never committed	28 tests; 7 manifests with Git state, module and notebook hashes; verifier; checkpoint replay	Kept and strengthened; now committed (`e06479c`)
Honest about limits	'Claims the study does not establish' section	'Evidence and limitations' and README 'Interpretation'; negative results kept	Kept; fix 3 garbled generated sentences
Correlation vs magnitude (Fig. 9)	Figure and narrative	Data, code and a regression test kept; figure dropped from the generated report	Restored here as Figure 5
Full vs computed views	Figs. 10-12, including CKA coverage	Both views x both designs in `comparison.csv`; PR-only line plot	Kept as data; limits noted

- **Reproducibility.** Rerun on 17 October 2026: 28 of 28 upgrade tests pass (56.7 s); the original suite passes 8 of 8 from a clean export of 2dc9118; `verify_results.py` passes; all 36 checkpoints replay, and they retrain byte-identically. Code and evidence are now committed together as `e06479c`. Caveat: the manifests were written before that commit, so they still record `2dc9118` with `git_dirty = true`. Their module hashes tie them to the code: the validation manifest matches all 9 modules in `e06479c`; the other six match 6 of 9 (`study.py` changed later; `research.py` and `report.py` did not exist yet). The six numerical-core modules are identical, and with the committed code the test suite regenerates programs, trace and geometry, the matched stage tables recompute to 3.6e-15 (6 programs), and all 36 trained models retrain byte-identically. The original's own module hashes were already stale. Fix: regenerate every reference from a clean checkout of `e06479c`, with the output outside the work tree so that `git_dirty` means something.
- **Honesty.** The original anticipated the variance diagnosis: it said total variance was not saved, named tiny numerical variation as one possible cause, and proposed recording it (original report, lines 447, 460, 510). To fix: three garbled generated sentences (`research-validation.md:11, 55, 143`). The generated prose reports only 1 of the 6 (out of 36) program x detector cells at or below always-positive. And `results/report/review.md` presents automated model audits as 'Independent review'; it should be labelled as automated.
- **Fig. 9.** The lesson is enforced by a test (`test_studies.py:155-168`: scaling lanes x2 keeps r = 1 while fraction_close < 1), and Section 5 makes it sharper. Correlation stays within 3.1e-12 of 1 while the magnitude check flips with attention sharpness.
- **Full vs computed.** Still informative: CKA is defined at 54/54 transitions in the full view and 24/54 in the computed view, and P12's PR error is 0.43 full vs 2.13 computed. But the computed view is defined by Tracr lane labels. It breaks under rotation (Section 9) and does not transfer to trained or InterpBench models without labels. Its substance has also changed: of the original contrasts, the P07/P08/P10 view gap and 12 of the 14 coverage mismatches were variance artifacts (Section 4); the contrasts that survive are P05, P06, P09 and P12.

15 What we withdraw or correct

#	Claim or text	Action
1	PR error as a headline metric (first column of original Table 1; `report.py:195-196`)	Withdraw as headline; report lane error and UVF; PR only normalized
2	A as causal evidence; 'All 1,280 conditions agree'	Withdraw; A is a readout check. Only 128 conditions exercise any downstream computation
3	Complexity analysis of the 12 programs (Fig. 13; Fig. 1 'different notions of complexity')	Withdraw; remove or annotate the notebook 05 complexity plot
4	Figs. 10-12 statements about P07, P08, P10 and P11	Withdraw (artifact); note that P11's design comparison reverses
5	'Same failing-probe set False' (`research-validation.md:143`)	Reword: failing sets are disjoint hot lanes
6	'variance between 8.06592e-19 and 8.06592e-19'; P11 omitted; L1/MLP variances cited	Correct: one value at 12 stages including P11; cite L1/Attn variances
7	'every branch contributes to the output' (complexity family)	Qualify: every branch is a function of the token alone and all 4-valued merges induce the same partition; at d = 2 branches compute the same function
8	'A ridge decoder maps the first three PCs to ... variables'	Qualify: only one compiler variable is a candidate at each stage
9	'Independent review' (`results/report/review.md`)	Relabel as automated model review
10	'PR and CKA recover some write timing' (upgrade change-report PDF, p. 1)	Restate: they recover non-empty sublayers, which can be read from the weights
11	'In 1 held-out program-detector comparisons...' (`research-validation.md:55`)	Correct: 6 of 36 cells (1 held-out, 5 calibration) do not exceed always-positive
12	`canonical_lane_agreement` (`rotations.csv`) read as a lane check	Relabel: it is a whole-residual element-wise comparison; the lane check is recomputed in Section 9

16 Experiments that could answer the headline question

Listed in priority order. Items 1-4 and 6 test whether PR or CKA reveal computation where some of Tracr's artifacts (empty sublayers, axis-aligned lanes, append-only storage) are removed while ground truth is kept. Item 5 fixes the complexity design; items 7 and 8 are numerical hygiene and provenance.

#	Experiment	What it tests	Falsifiable prediction
1	Tracr models with learned compression: train only the projection (Tracr Section 5), then run the full benchmark and blind recovery	Superposition with known ground truth (empty sublayers stay empty)	If PR and CKA track computation, gaps and detection degrade only as accuracy degrades
2	Erase-and-reuse compiler pass	Lifetimes and overwriting	Variance or CKA drops flag erasures; a detector tells erasures from writes
3	InterpBench SIIT models, threshold-free scoring against `edges.pkl`	Dense trained weights with known circuits	Detectors fall to chance if they only exploit empty sublayers
4	Perturbed schedules: inserted non-empty no-op sublayers, delayed writes; end-to-end fine-tuning of compiled weights	Whether detections follow the schedule	Predicted write stages move with the inserted shift
5	Redesigned complexity family (depth decoupled from branch count via balanced merge trees, crossed with cardinality; attention hops; 3-5 replicates per cell)	Depth vs width vs cardinality	Pre-specified ANOVA separates the three factors
6	Patching at every CKA-ranked transition; pivotal-pair sampling; DAS directions; trained models	Whether geometry finds causal variables	PCA/DAS beats random at downstream sites, not just readout sites
7	Numerical hygiene: pad the last batch; scale-relative or 1e-8 floor; UVF; count scratch writes in blind scoring	Robustness	No conclusion changes; noise at the source drops to 0
8	Regenerate all references from a clean checkout of `e06479c`, with `--output` outside the work tree, then copy them in	Provenance	Every manifest records `git_dirty = false` and the same module hashes

Appendix A Reproduction checks run for this response

Check	State	Result	Time
`unittest discover -v`	Upgrade (tree later committed as `e06479c`)	28/28 OK	56.7 s
Pushed commit `e06479c` vs the analysed tree	Upgrade	Identical in code and `results/<study>/reference/`; the commit adds README links and the two PDFs under `output/pdf/`. Local run folders (`results/*/runs/`) are gitignored and not committed	-
`unittest discover -v` on a clean `git archive 2dc9118`	Original	8/8 OK	35.7 s
`scripts/verify_results.py` (on a byte copy of the evidence)	Upgrade	Pass: 7 manifests, 12 programs, 24 prediction hashes, 36 checkpoint hashes	~1 s
Read-only checkpoint replay	Upgrade	36/36 predictions, targets, accuracies and SHA-256 match	11.8 s
Full retrain of the 36 trained models	Upgrade	36/36 `model.npz` byte-identical; predictions and training logs identical	105 s
Stage tables recomputed with current code	Upgrade	Matched tables for P05-P08, P10, P11: within 3.6e-15; an independent reimplementation over all 412 rows: within 7.8e-9	-
Notebook execution; full study rerun	Upgrade	Not rerun; the 15 October record lists 5/5 notebooks passing	-

Apple M5 Pro CPU, Python 3.12.14, jax 0.11.2, Tracr 9ce2b8c. Nothing under `results/` was modified; tools that write there were redirected to copies.

Appendix B Evidence index

Section	Repository evidence (upgrade)	Analysis for this response
3	`results/comparison/reference/{matched,balanced}/`; `results/alignment/reference/`	`circularity-pr/`
4	`results/comparison/reference/*/variance-sensitivity.csv`, `stages.csv`	`numerics/`
5	`results/validation/reference/p12-sorting/`	`p12/`
6	`results/programs/reference/`; `results/validation/reference/complexity/`	`complexity/`
7	`results/alignment/reference/interventions.csv`; `results/validation/reference/*/patches.csv`	`interventions/`
8-10	`results/validation/reference/*/{rotations,nonorthogonal,compression}.csv`	`transfer/`
11-12	`results/learning/reference/`; `results/validation/reference/*/write-recovery-scores.csv`	`learning-blind/`
13-14	`results/report/research-validation.md`; `README.md`	`context-repro/`

Analysis folders are under `.context/review-response/`, which is gitignored and therefore not in `e06479c` or on the remote; they are available on request. Each has a `verify/` subfolder holding the independent re-check scripts and their outputs.

References

[1] Gupta, R., Arcuschin, I., Kwa, T., Garriga-Alonso, A. *InterpBench: Semi-Synthetic Transformers for Evaluating Mechanistic Interpretability Techniques.* NeurIPS 2024 Datasets and Benchmarks. arXiv:2407.14494.

[2] Lindner, D., Kramar, J., Farquhar, S., Rahtz, M., McGrath, T., Mikulik, V. *Tracr: Compiled Transformers as a Laboratory for Interpretability.* NeurIPS 2023. arXiv:2301.05062.

[3] Weiss, G., Goldberg, Y., Yahav, E. *Thinking Like Transformers.* ICML 2021, PMLR 139:11080-11090.

[4] Kornblith, S., Norouzi, M., Lee, H., Hinton, G. *Similarity of Neural Network Representations Revisited.* ICML 2019, PMLR 97:3519-3529.

[5] Gao, P., Trautmann, E., Yu, B., Santhanam, G., Ryu, S., Shenoy, K., Ganguli, S. *A theory of multineuronal dimensionality, dynamics and measurement.* bioRxiv 214262, 2017.

[6] Elhage, N., et al. *Toy Models of Superposition.* Transformer Circuits Thread, 2022. arXiv:2209.10652.

[7] Friedman, D., Wettig, A., Chen, D. *Learning Transformer Programs.* NeurIPS 2023. arXiv:2306.01128.

[8] Conmy, A., Mavor-Parker, A. N., Lynch, A., Heimersheim, S., Garriga-Alonso, A. *Towards Automated Circuit Discovery for Mechanistic Interpretability.* NeurIPS 2023. arXiv:2304.14997.

[9] Geiger, A., Wu, Z., Lu, H., Rozner, J., Kreiss, E., Icard, T., Goodman, N., Potts, C. *Inducing Causal Structure for Interpretable Neural Networks.* ICML 2022, PMLR 162:7324-7338.

[10] Geiger, A., Wu, Z., Potts, C., Icard, T., Goodman, N. D. *Finding Alignments Between Interpretable Causal Variables and Distributed Neural Representations.* CLeaR 2024, PMLR 236:160-187.

[11] Thurnherr, H., Scheurer, J. *TracrBench: Generating Interpretability Testbeds with Large Language Models.* arXiv:2409.13714, 2024.